



Embedded Development Using Zephyr & Open Source Tooling

Kevin Townsend
Maureen Helm

analog.com



Meet the Presenters

PRESENTER

Maureen Helm

Distinguished Engineer, Software Architect

Github: @MaureenHelm



PRESENTER

Kevin Townsend

Distinguished Engineer, Tools Architect

Github: @microbuilder



Today's Agenda

- Introduction 5 mins
- CodeFusion Studio 10 mins
- Lab 1: Hello World 20 mins
- Zephyr Build System and Configuration 15 mins
- CodeFusion Studio Config Tools 15 mins
- Lab 2: Blinky and Devicetree Overlays 10 mins
- Break 15 mins
- Lab 3: Debug 30 mins
- Instrumentation and Profiling Tools 20 mins
- Zephyr Subsystems and Drivers 10 mins
- Lab 4: Sensors, Shell, and Shields 15 mins
- Wrap Up 10 mins

About the Hardware

[AD-APARD32690-SL](#)

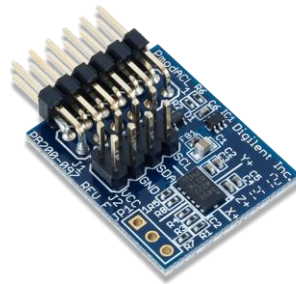
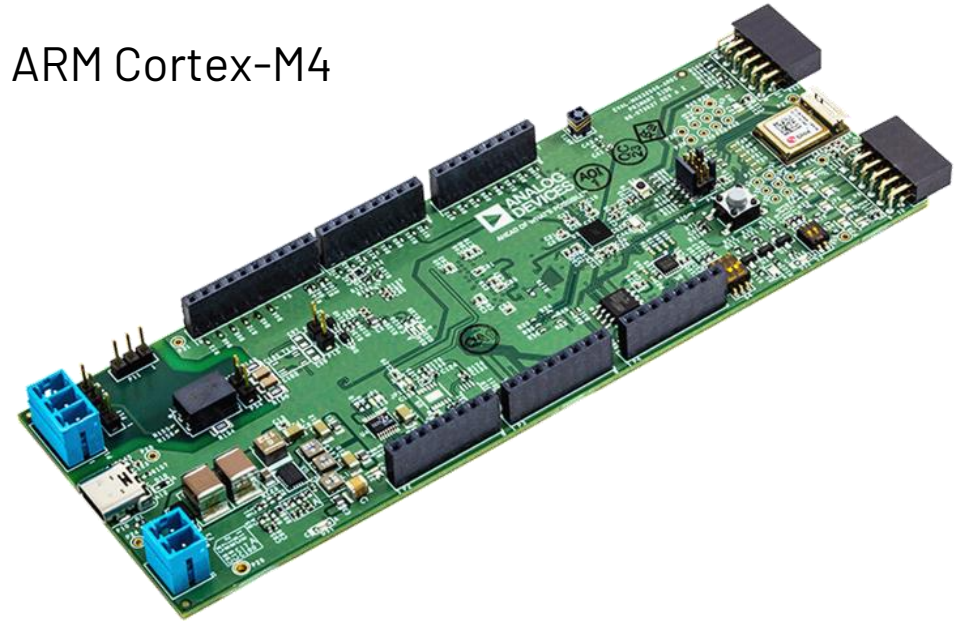
- Arduino Form-factor Development Platform Based on MAX32690 ARM Cortex-M4 Microcontroller with RISC-V coprocessor
- Main development board

[MAX32625PICO](#)

- Evaluation Kit for the MAX32625/MAX14750
- Debug probe

[Pmod ACL](#)

- ADXL345 3-axis Accelerometer
- Shield board



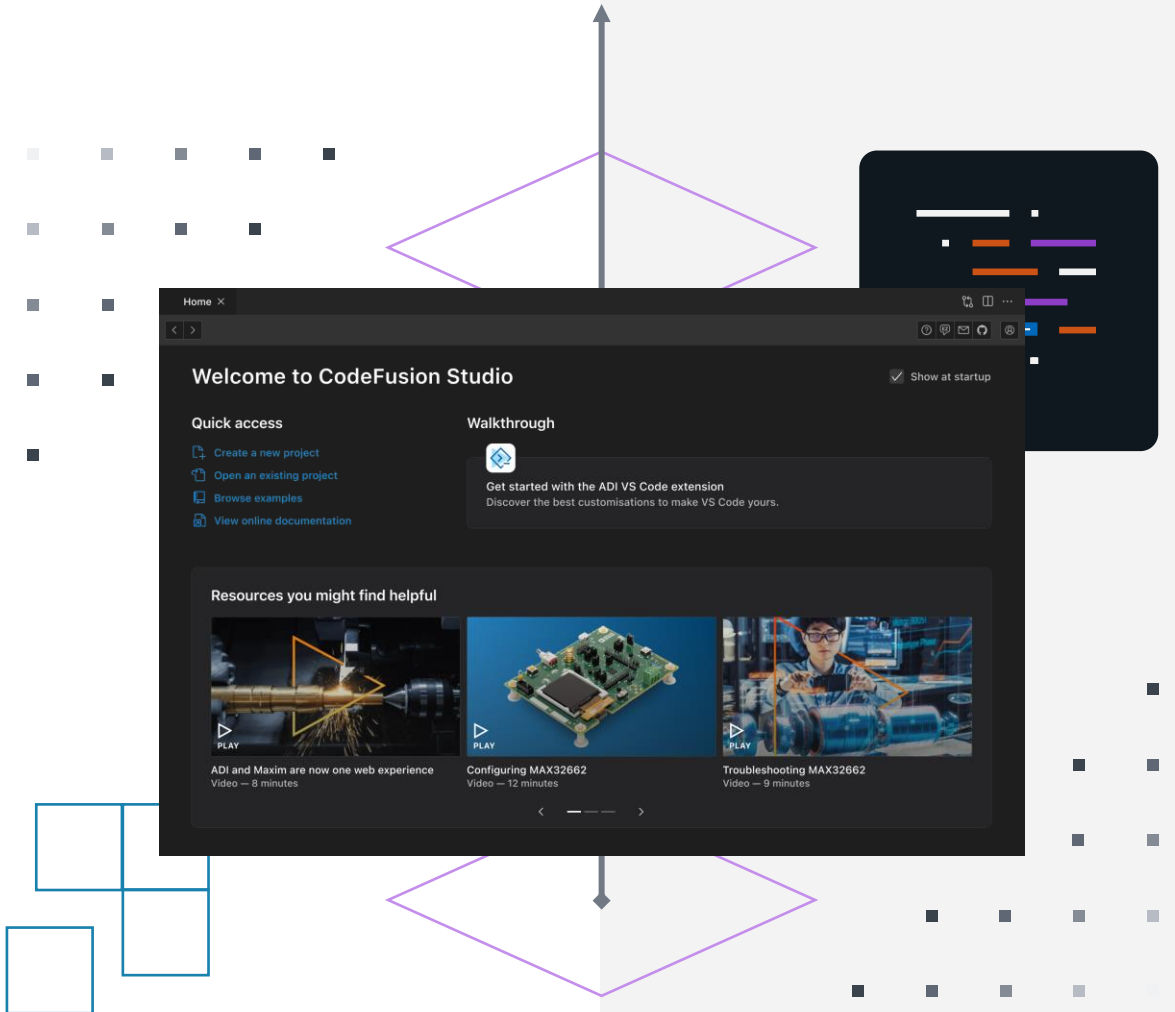
The background of the image is a solid blue color on the left side, which transitions into a pattern of dark blue and light blue triangles on the right side. The triangles are arranged in a way that creates a 3D effect, resembling a stack of cubes or a geometric pattern.

CodeFusion Studio®



CodeFusion Studio

Built from the ground up
to simplify the embedded
developer experience



Design Principles: Open Source First

Strategic preference at ADI for:

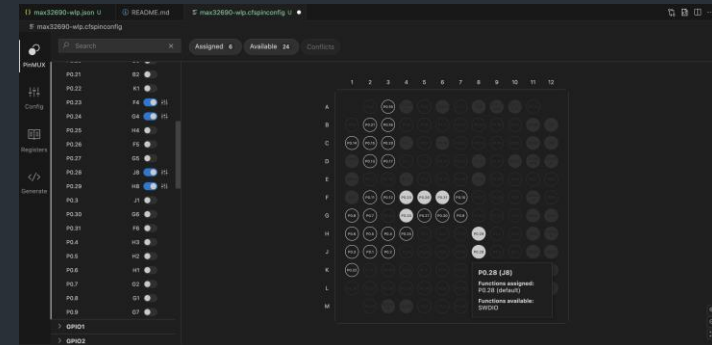
- Open source toolchains
- Open source tools
- Open source middleware from active communities
- **Tools can be freely extended** by users for their niche needs
- **Code generation system is plugin-based**, and can be freely extended

FEATURE

Open Source Config Tools

Standard config tool GUIs -- pin config, clock config, etc -- are implemented as React applications, hosted inside Visual Studio Code.

These cross-platform tools can be modified using a framework many devs are already familiar with.



Design Principles: Command-Line First

- A large part of embedded development happens on the command-line
- CodeFusion Studio is built around a single command-line utility (**cfsutil**) that is used to:
 - Enable new SoCs to be supported in the IDE
 - Generate config code (pin/clock config, etc.)
 - Manage toolchain of software dependencies
 - Provide a CI-friendly workflow
- This CLI tool has a **plugin architecture**:
 - New commands can easily be added to it
 - New export targets for code-gen tools can be written (adding a new RTOS, etc.)

FEATURE

cfsutil (Command-Line Utility)

As much processing logic as possible has been placed on the command-line.

Key features like config code generation can easily be integrated into CI build or test pipelines.

```
~/adi/node/scs-scsutil/packages/cli $ yarn cfsutil generate --help
Generate source code from the provided .cfspinconfig file.

USAGE
$ cfsutil generate -i <value> [-e <value>] [-o <value>] [-v] [-p] [-f
text|json] [--force] [--list] [--file <value>]

FLAGS
-e, --engine=<value> [default: baremetal] Set the code generation engine.
-f, --format=<option> [default: text] Set the output format for preview.
    <options: text|json>
-i, --input=<value> (required) Set the .cfspinconfig file.
-o, --output=<value> [default: .] Set the output directory for generated code.
-p, --preview Display generated code on stdout.
-v, --verbose Display full paths for generated code.
    --file=<value> Generate only the specified file.
    --force Forces file overwrites and folder creation.
    --list List the filenames that will be generated by the generate
command.

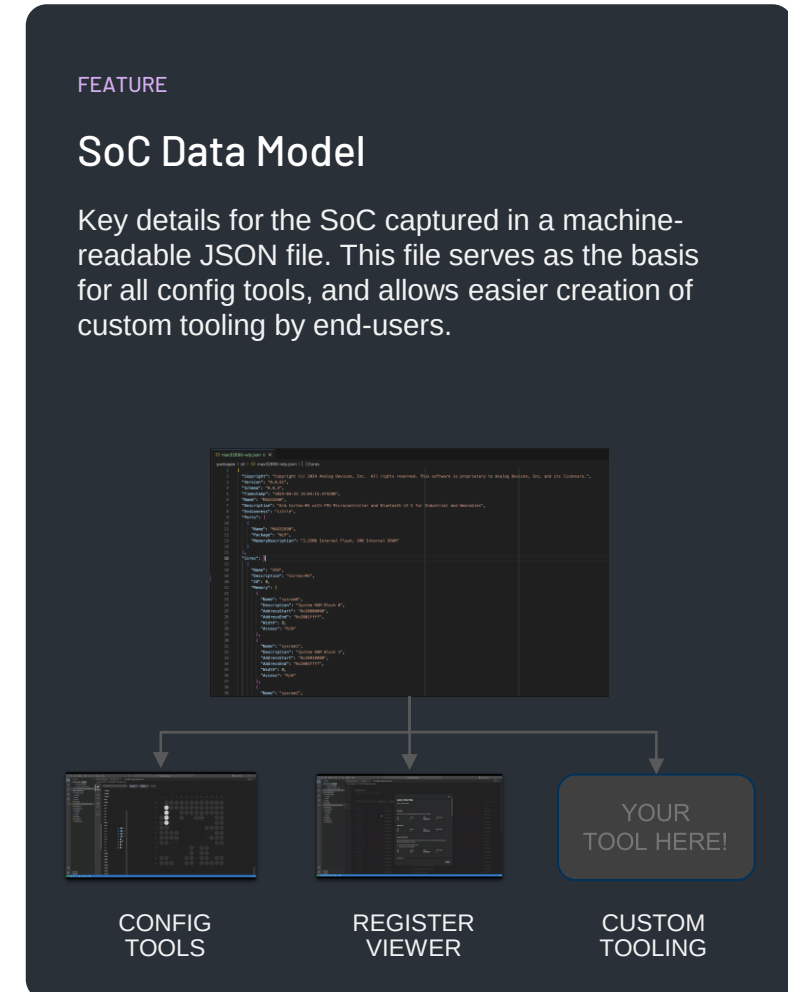
DESCRIPTION
Generate source code from the provided .cfspinconfig file.
```


Design Principles: Data-Driven Config Tools

All tools in CodeFusion rely on a machine-readable JSON file called the **SoC Data Model**.

This file is used by all config, debug and profiling tools in the IDE, and contains:

- Details on individual processing domains or cores
 - Memory
 - Architecture
- Package information
- Pin multiplexing and config data
- Clock source and config details
- Peripheral details (UART, I2C, etc.)
- Register details



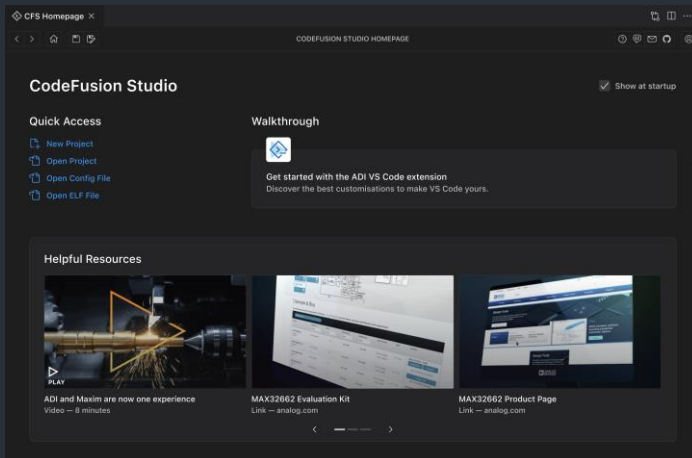


Key Features – 1.0

FEATURE

Quick Access to Docs & Resources

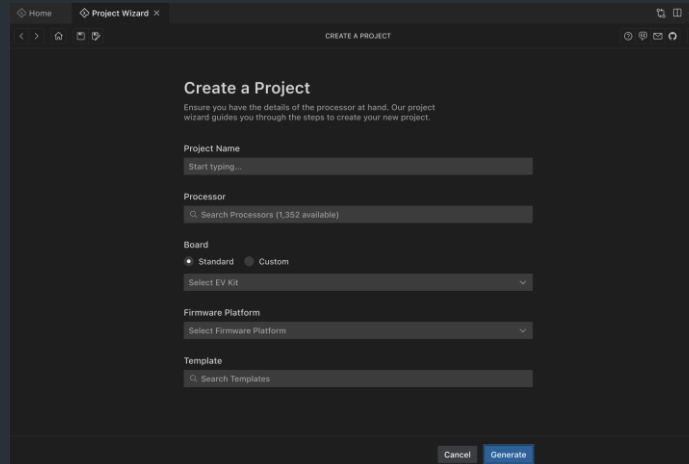
Ready access links for common tasks. Links to articles and videos related to your projects, user guides, hardware reference manuals, data sheets, and other useful resources all in one easily accessible space.



FEATURE

New Project Wizard

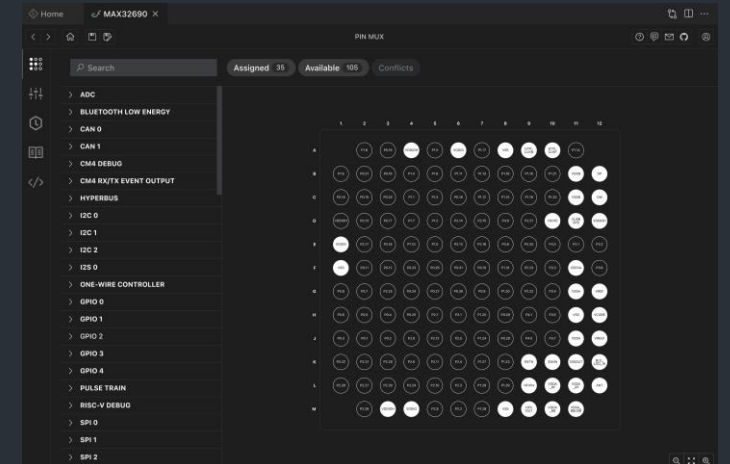
Quickly create projects using example applications to jumpstart your development tool chain.



CONFIGURATION TOOL

Pin Configuration

Assign signals to pins, configure pin values such as input or output mode and power supply, view register details and values, and generate source code to be included in your project.



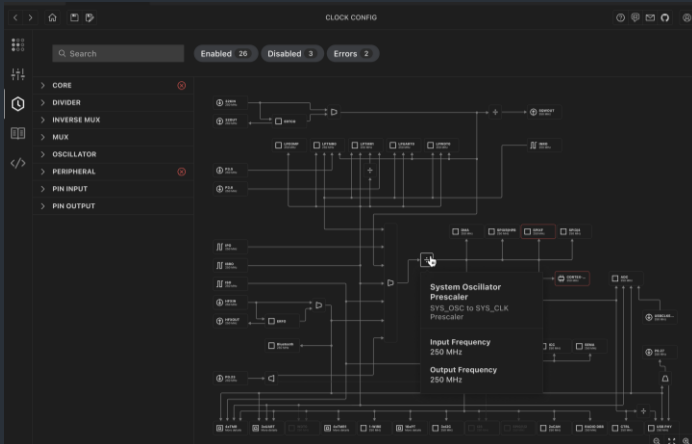


Key Features – 1.0

CONFIGURATION TOOL

Clock Configuration

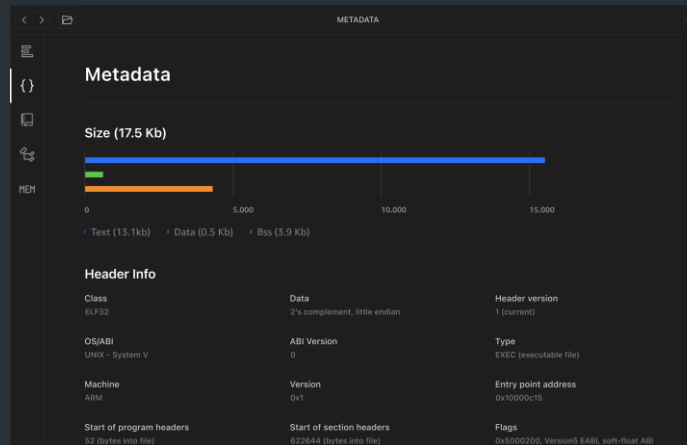
Graphically configure the clock tree for your SoC using an interactive schematic view. Selectively enable and disable peripheral blocks, previewing the clock frequency for each node or trace in the schematic, and generate code to implement those config choices.



FEATURE

ELF File Explorer

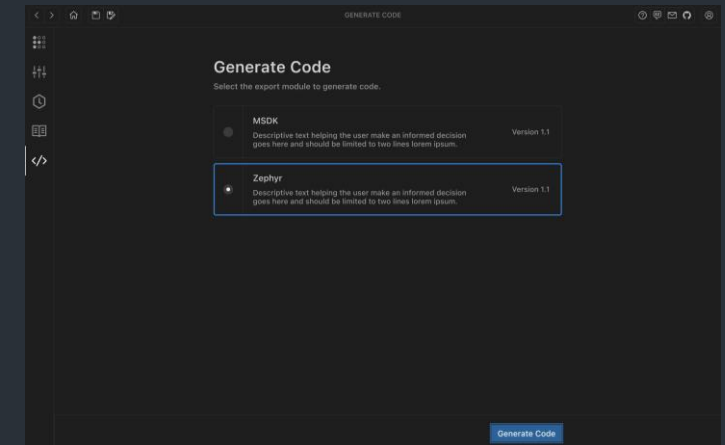
Save hours by running SQL queries against firmware images for resource placement, memory usage, and compiler settings in a convenient graphical tool versus manually collating results from numerous command line utilities.



FEATURE

Code Export

Separate config choice capture from code generation with an extensible, plugin-based code generation engine. Out of the box support included for MSDK and Zephyr 3.7.





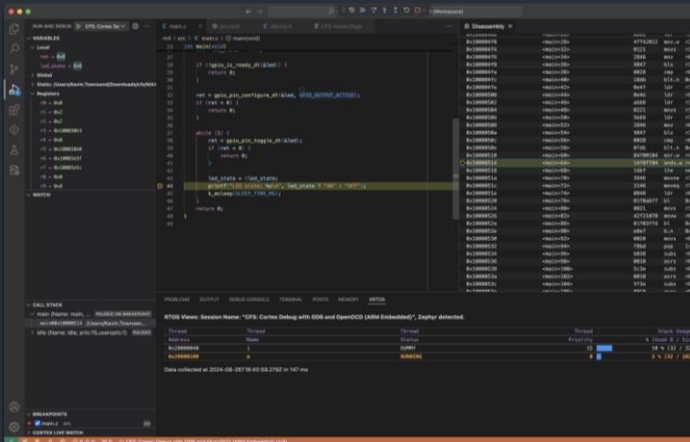
Key Features – 1.0



FEATURE

Multi-Core Debug

Essential debugging features including breakpoints, disassembly, heterogeneous debug, and RTOS thread awareness.



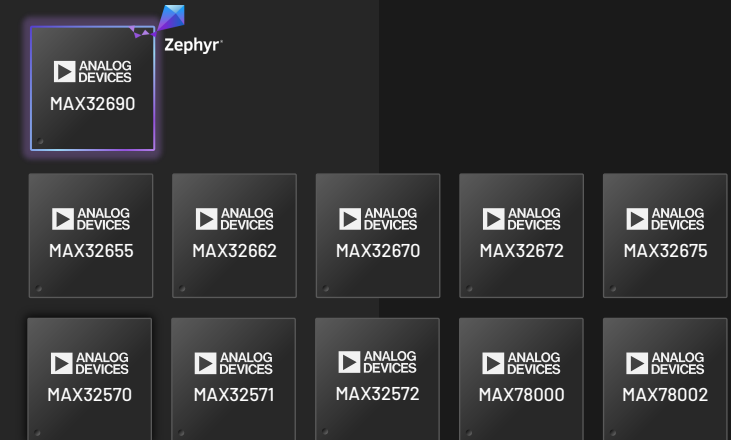
PART SUPPORT

Additional parts coming soon

CodeFusion Studio 1.0 supports:

Zephyr MAX32690, MSDK MAX

[MAX32655](#), [MAX32662](#), [MAX32670](#), [MAX32672](#),
[MAX32675](#), [MAX32690](#), [MAX32570/1](#), [MAX32572](#),
[MAX78000](#), [MAX78002](#)



STANDARD SUPPORT

ARM & RISC-V Support

Tools, libraries and support for developing software based on arm and RISC-V architectures.



STANDARD SUPPORT

Zephyr Support

Including project creation, build and debug.



Download Links

CodeFusion Studio

<https://developer.analog.com/solutions/codefusionstudio>



MAX32690 Development Board Details

<https://developer.analog.com/solutions/max32690>



Lab 1: Hello World

Lab 1: Hello, World!

Objectives

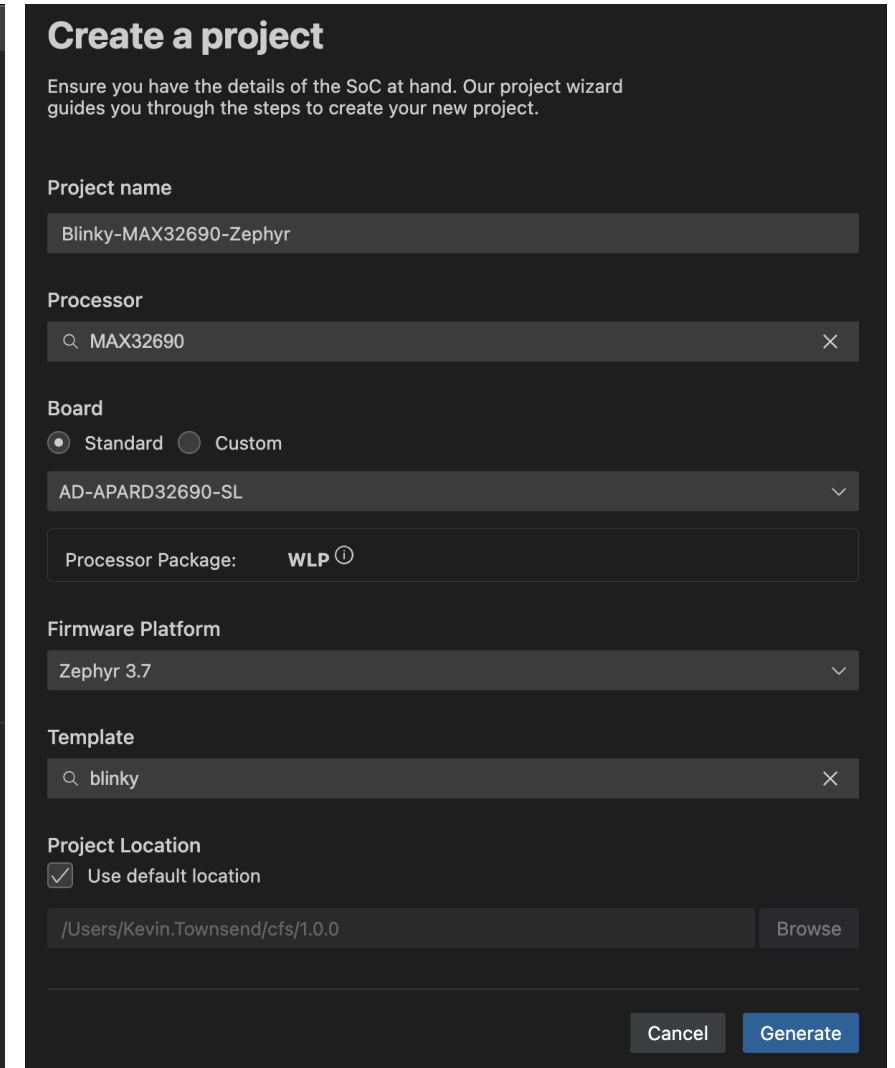
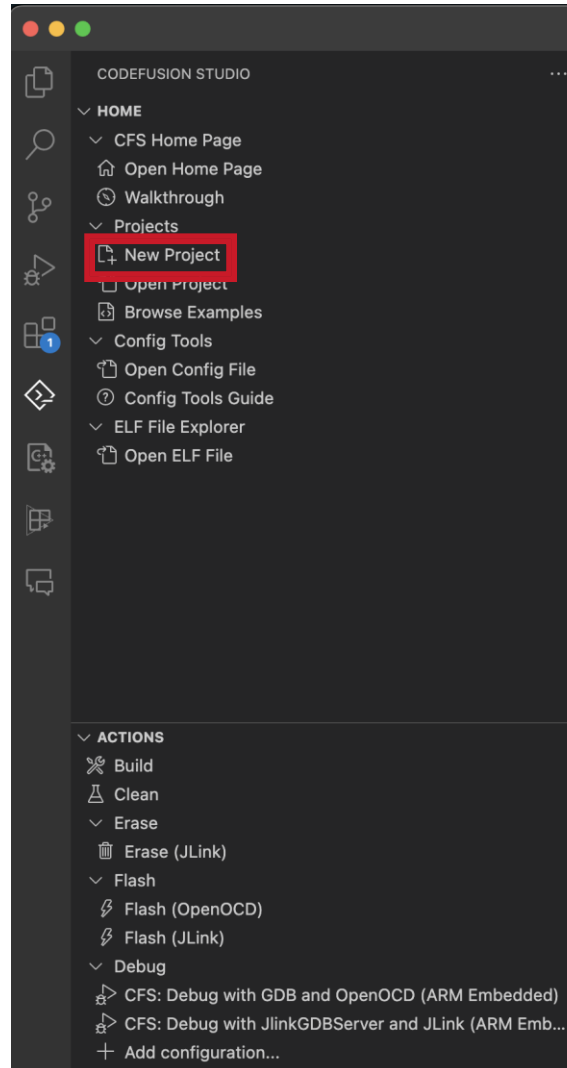
- Generate a simple project
- Familiarize yourself with the basic features of the IDE: Build, flash, view serial output
- View the ELF file output in the IDE

Generate a New Project

1. Switch to the CodeFusion Studio Panel

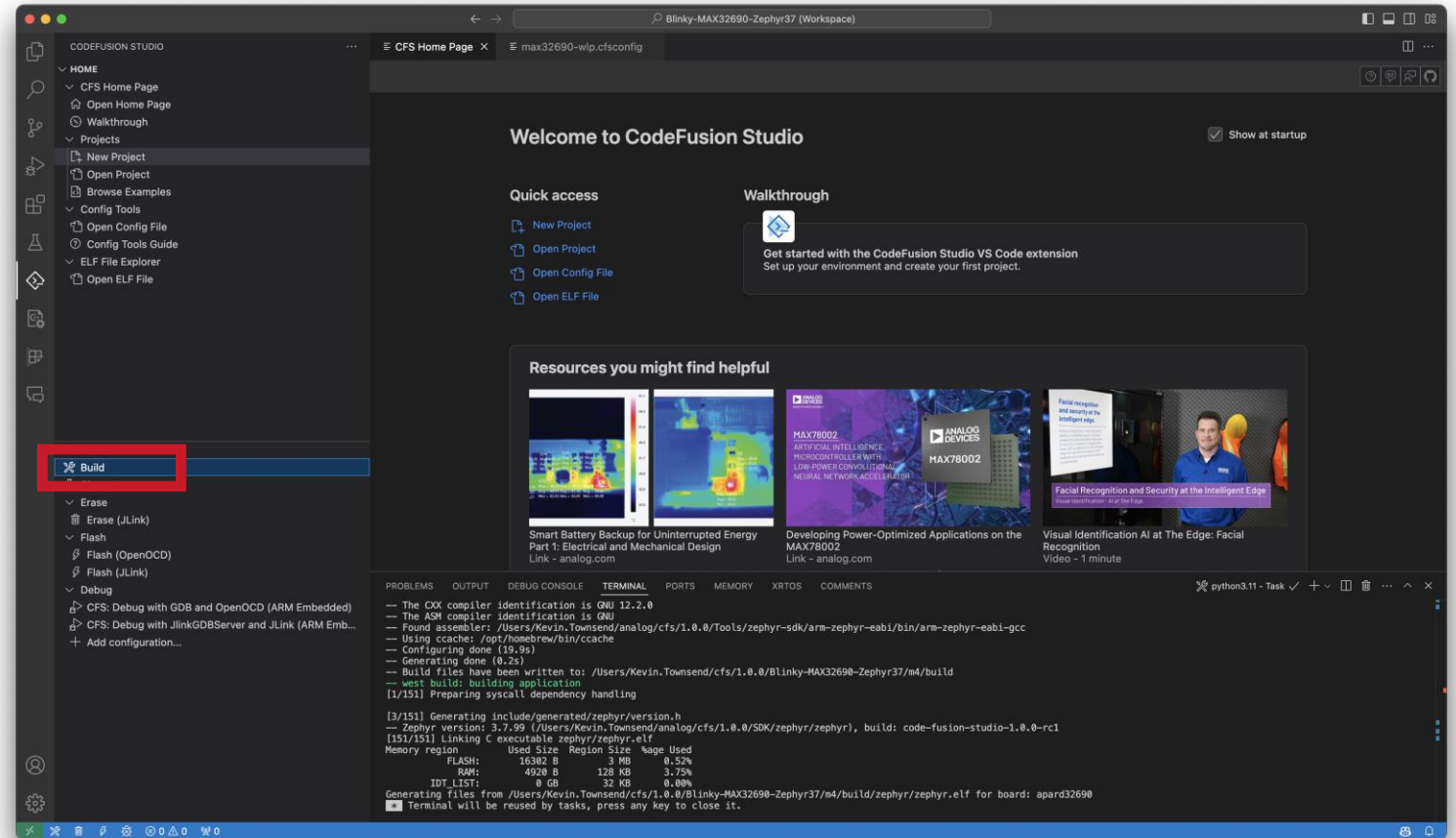


2. Select **New Project**
3. Enter a Project Name: **MAX32690**
4. Select the board: **AD-APARD32690-SL**
5. Select a platform: **Zephyr 3.7**
6. Selection a Template: **Blinky**
7. Click **Generate**.



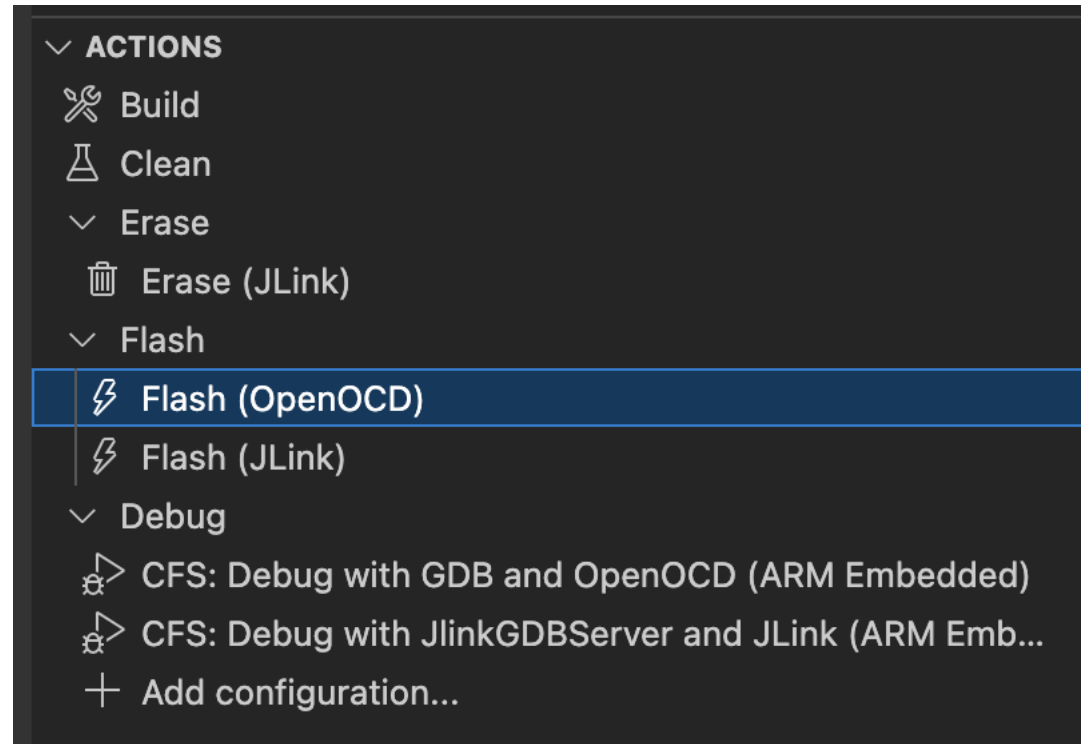
Build the Sample

1. Click the **Build** quick action
2. This will kick off a **west build** action, which you'll see in the terminal window at the bottom



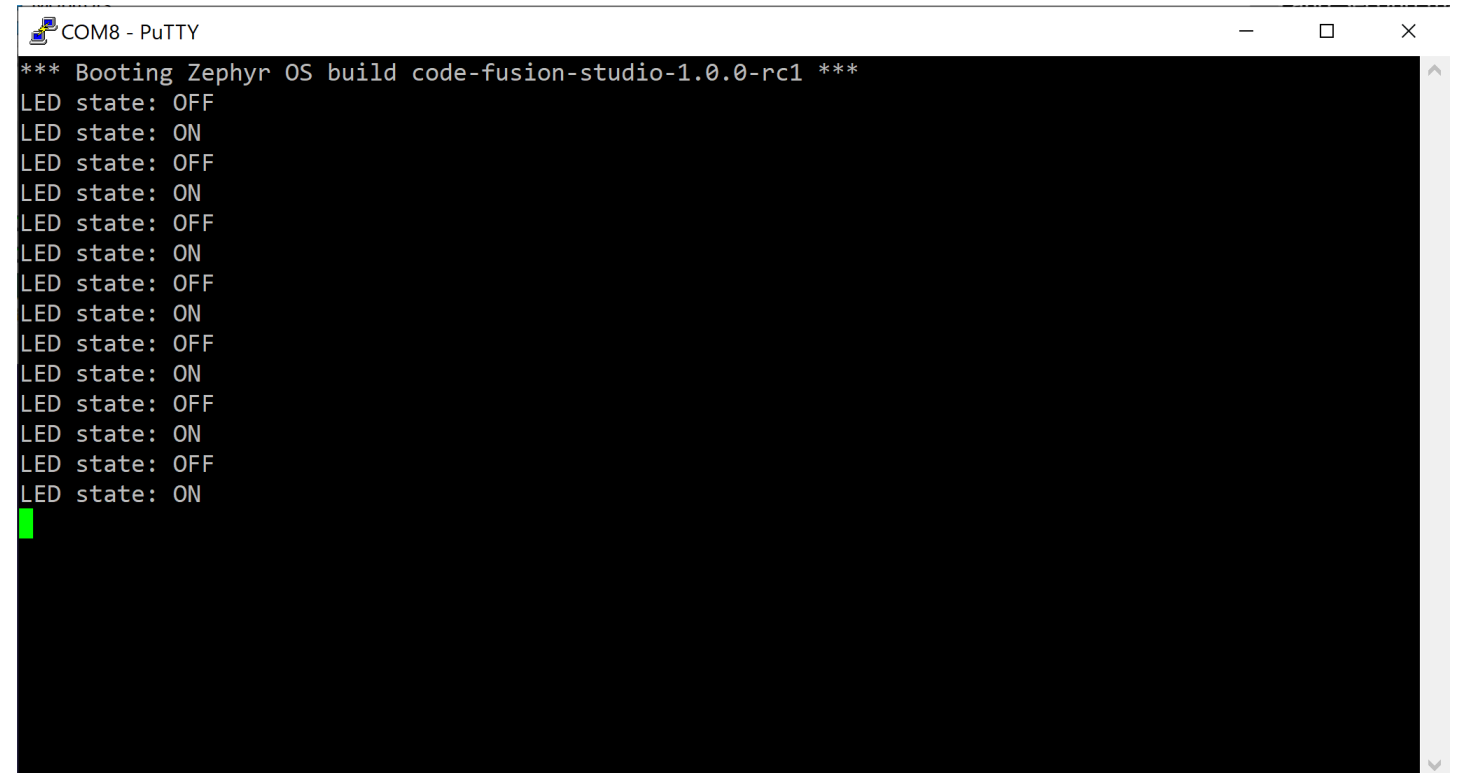
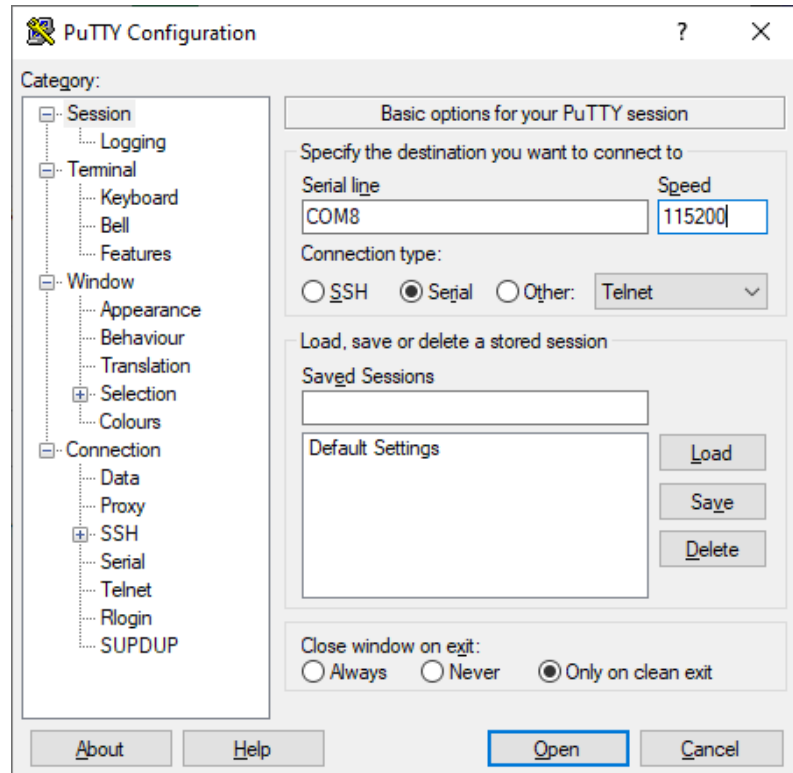
Flash the Sample to the Board

1. Click the **Flash (OpenOCD)** action to flash the sample to the board using the debugger.
2. Once this is done, you should see blinky on the PCB using the Blue LED.



View the Serial Output with Putty

You can preview the serial output with Putty selecting **Serial**, then the right **COM Port** and **baud rate** (115200):



Lab 1: Create, Build and Flash Blinky

Create a New Project

1. Switch to the CodeFusion Studio Panel



2. Select "**New Project**"
3. Enter a Project Name: **MAX32690**
4. Select the board: **AD-APARD32690-SL**
5. Select a platform: **Zephyr 3.7**
6. Selection a Template: **Blinky**
7. Click **Generate**

Build and Flash

1. Click the **Build** action
2. Once complete, click the **Flash (OpenOCD)** action
3. Make sure the Blue LED is blinking

To preview serial output:

1. Open Putty and select **Serial Port**
2. Set COM4 (etc.) and 115200 baud
3. Click 'Connect'

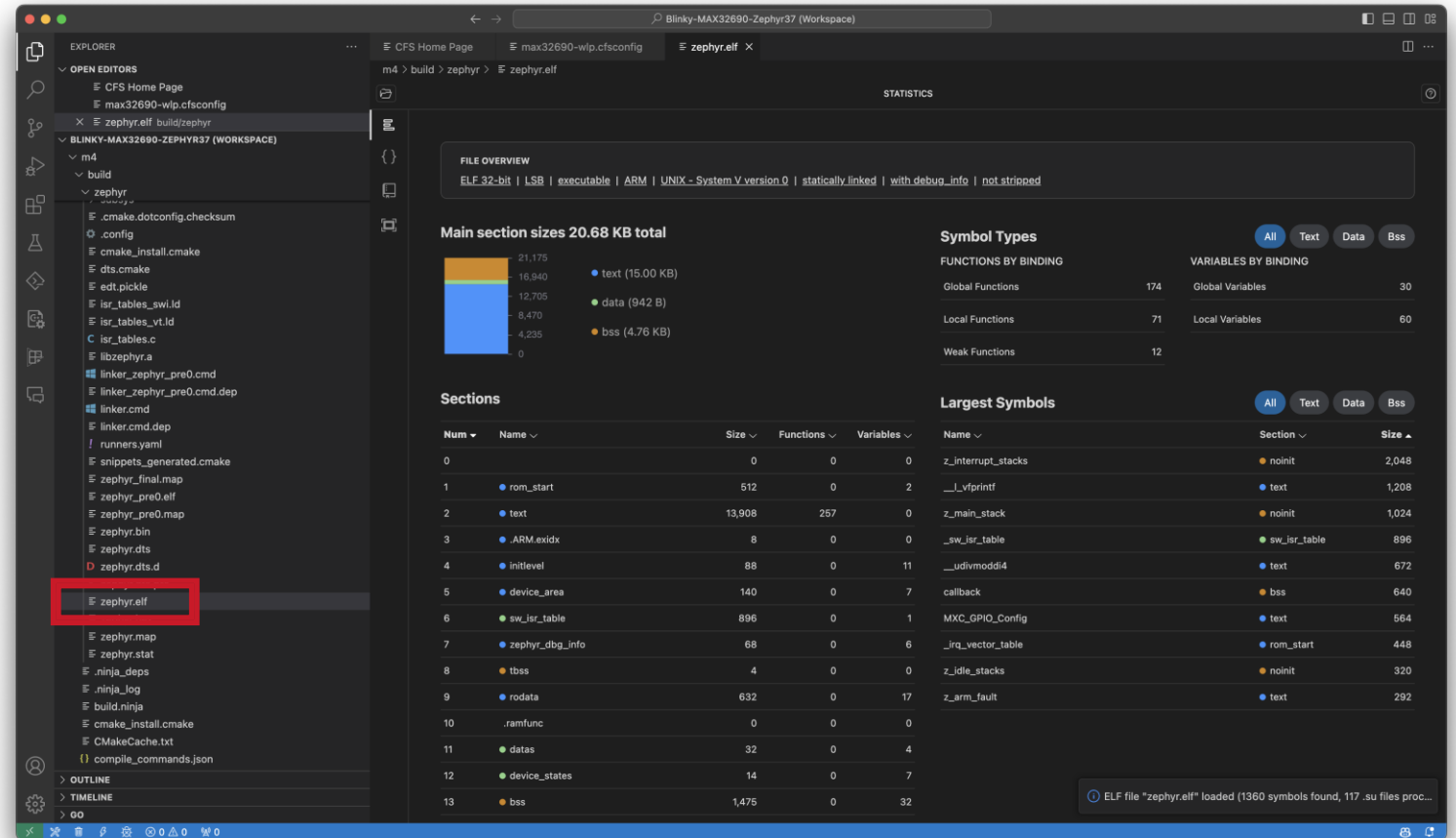
Bonus: ELF File Explorer

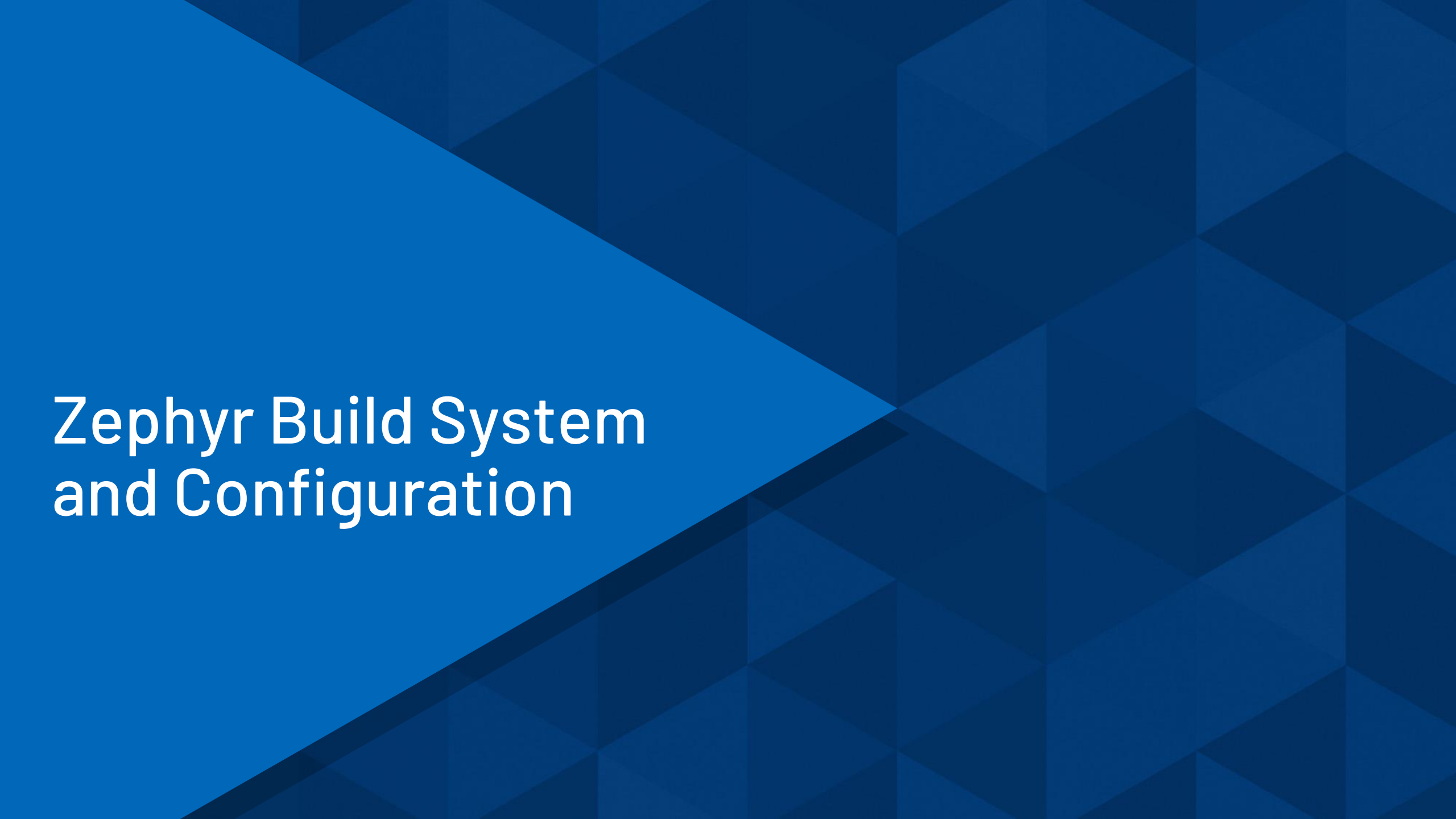
Navigate to **build/zephyr/zephyr.elf** and double-click the file to preview the ELFimage

Bonus: Preview Zephyr.elf in ELF File Explorer

1. Switch to the **Explorer** tab
2. Navigate to **build/zephyr/zephyr.elf**
3. Double-click on the .elf file

CodeFusion Studio includes a graphical ELF visualisation tool that you can use to interact with your compiled code, including running SQL queries to locate symbols precisely.





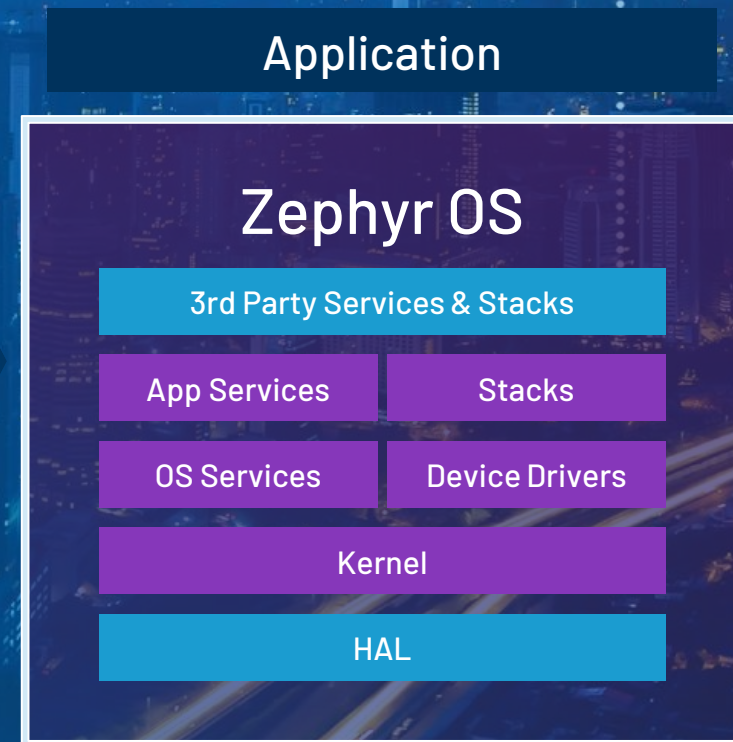
Zephyr Build System and Configuration

The Zephyr® Project

Open Source, RTOS, Connected,
Embedded fit where Linux is too big

- **Open-source** real time operating system
- **Developer friendly** with vibrant community participation
- Built with **safety and security** in mind
- **Broad SoC, board and sensor support**
- **Vendor Neutral** governance
- **Permissively licensed** - Apache 2.0
- **Complete**, fully integrated, highly configurable, **modular** for **flexibility**
- **Product** development ready using LTS includes security updates
- Project started 7+ years ago

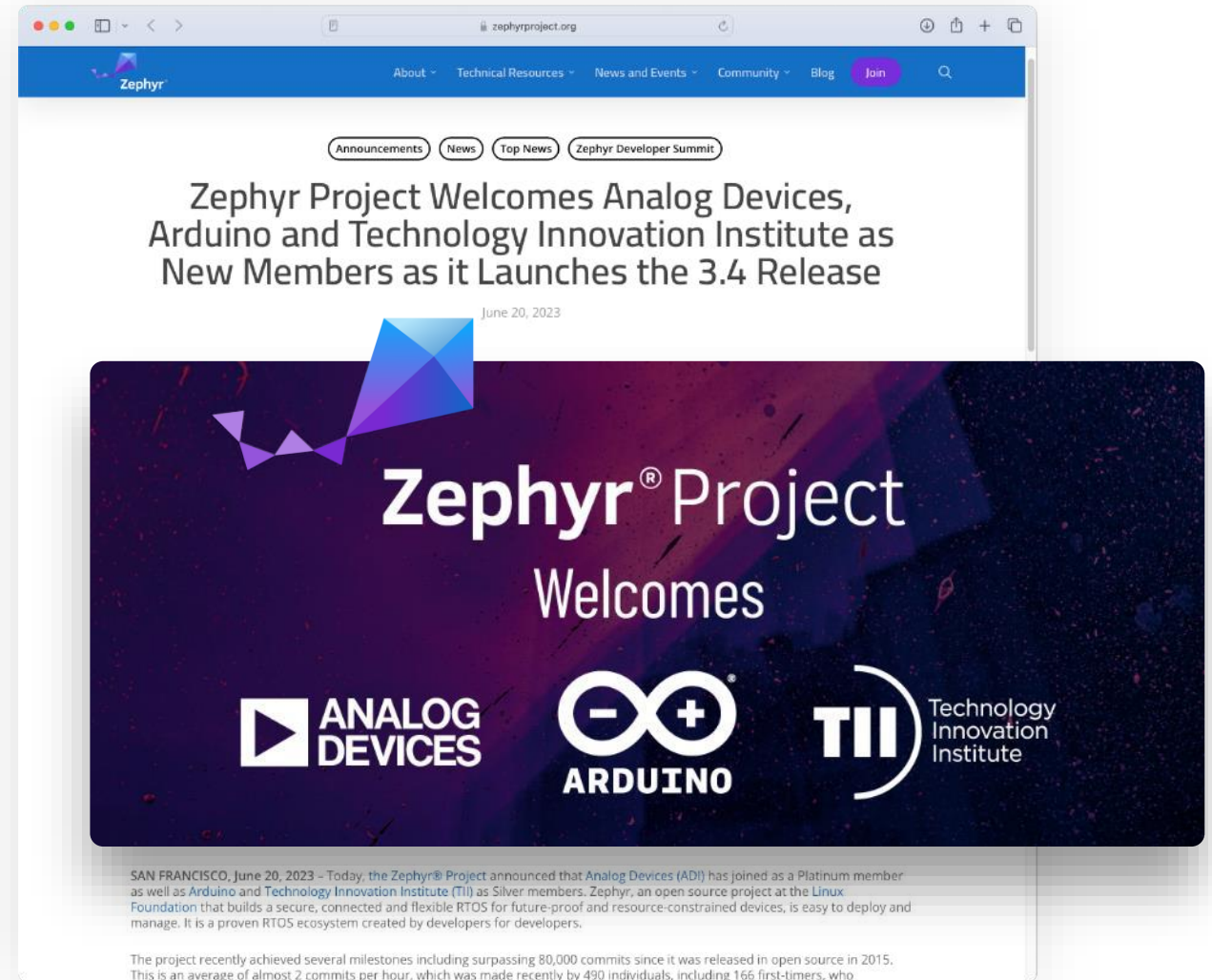
 THE **LINUX** FOUNDATION



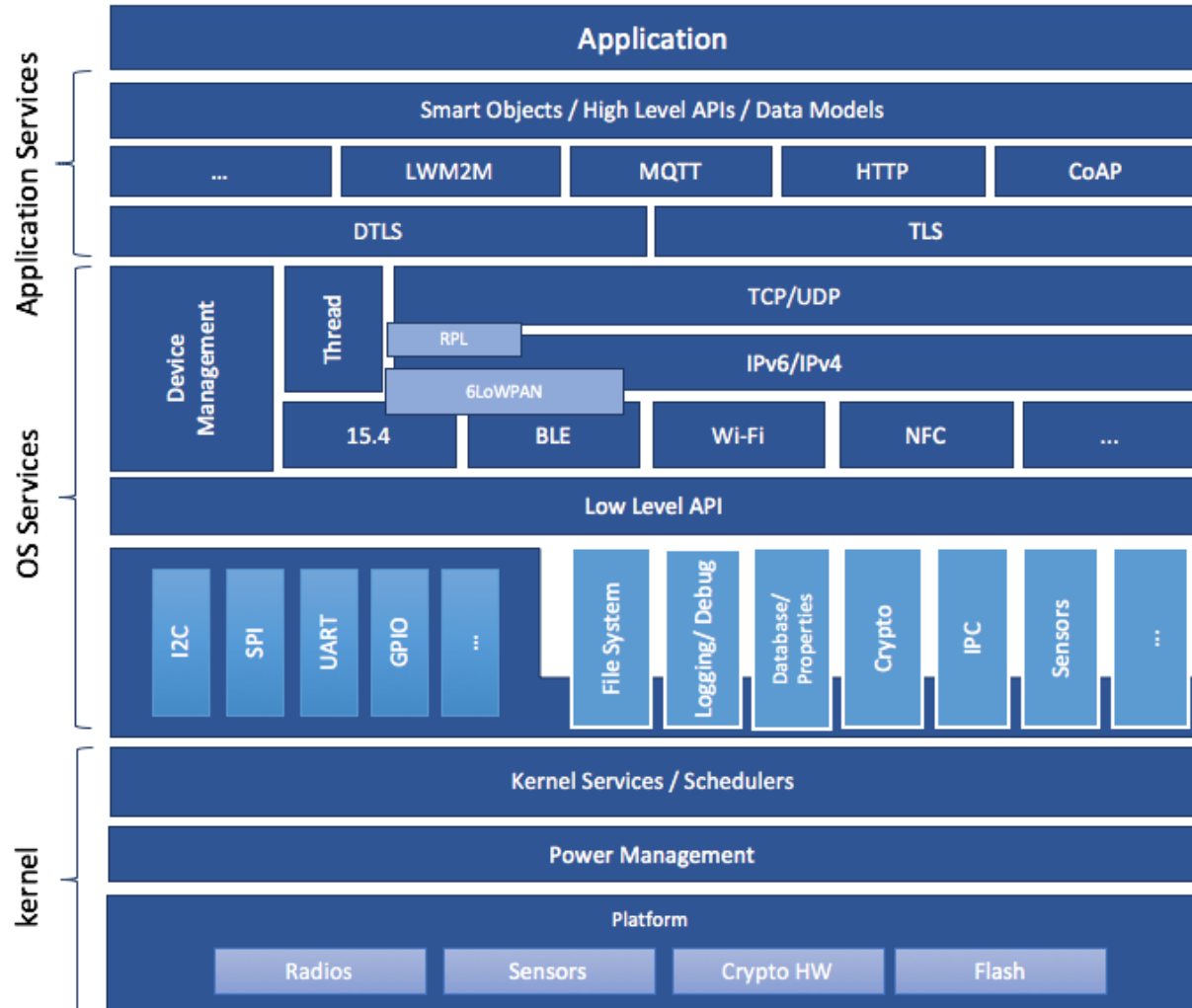
ADI's Strategic Support of Zephyr

At ADI, we are proud to support the Zephyr Project and join the vibrant ecosystem they have cultivated. Embedded software plays a pivotal role in delivering industry-leading digital products to our customer base, which exceeds 125,000 across new and emerging markets. By contributing to Zephyr, we continue to show our commitment to the open-source community and to our customers.

– Rob Oshana



The Zephyr® Project



Source: <https://docs.zephyrproject.org/latest/security/security-overview.html>

Highly Connected

- Ethernet, WiFi, Bluetooth BLE, USB, CANBus,...

Native TCP/IP Networking Stack

- Support for key Ethernet protocols

Real-Time Performance

- Deterministic, low latency compute & network

Resource Efficient

- Optimised for resource-constrained devices

Open Source

- Transparent, extensible software

Portability

- Support for a range of platforms and arch's

Security

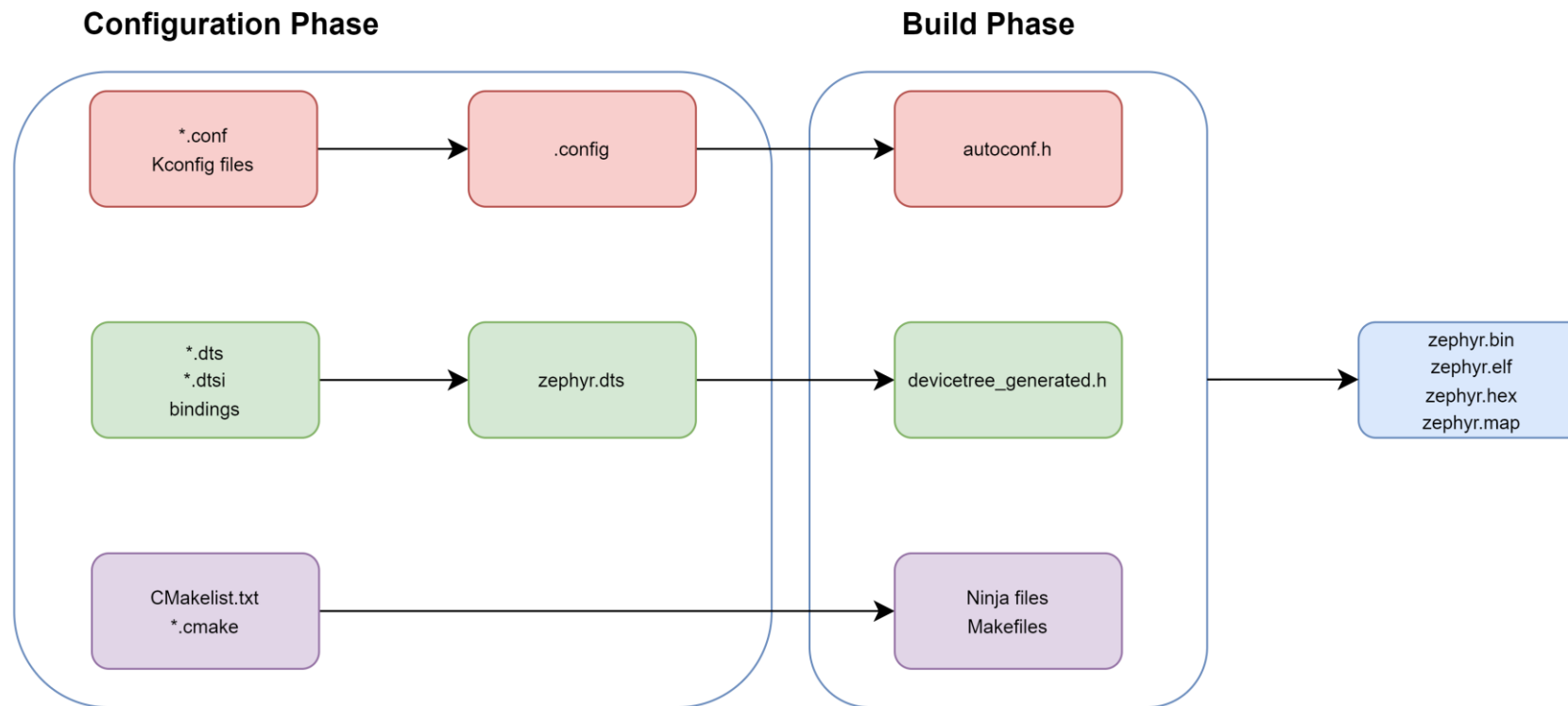
- Encryption, authentication, secure boot

Zephyr Build and Configuration Systems

Build System (CMake)

- Configuration Phase
 - [Devicetree](#): specifies the **hardware** configuration
 - [Kconfig](#): specifies the **software** configuration
 - Generates build system (Makefile/Ninja), preprocessor macros, ISR tables, syscalls
- Build Phase
 - Invokes the C compiler and linker

Zephyr Build and Configuration Systems

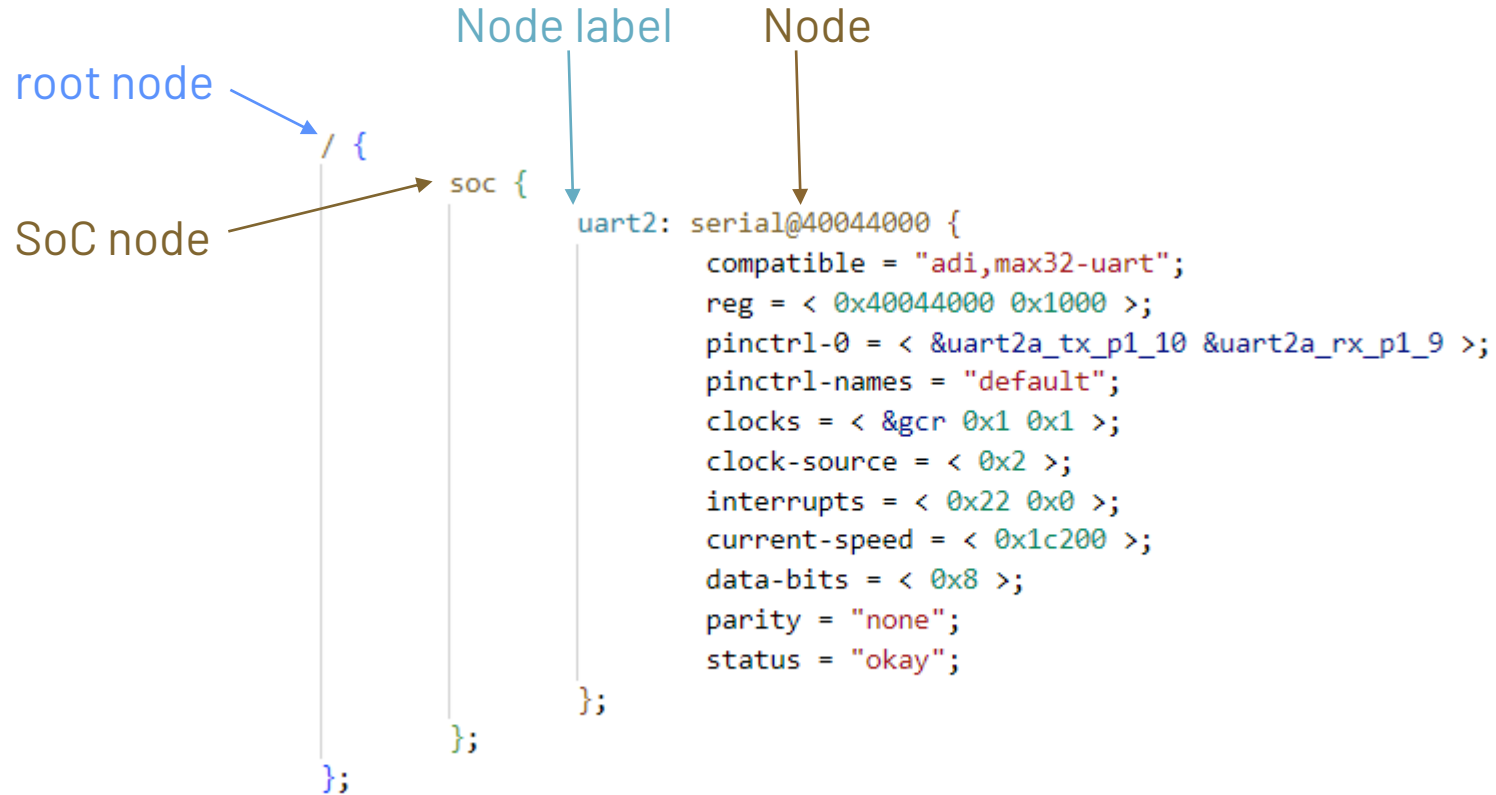


Devicetree

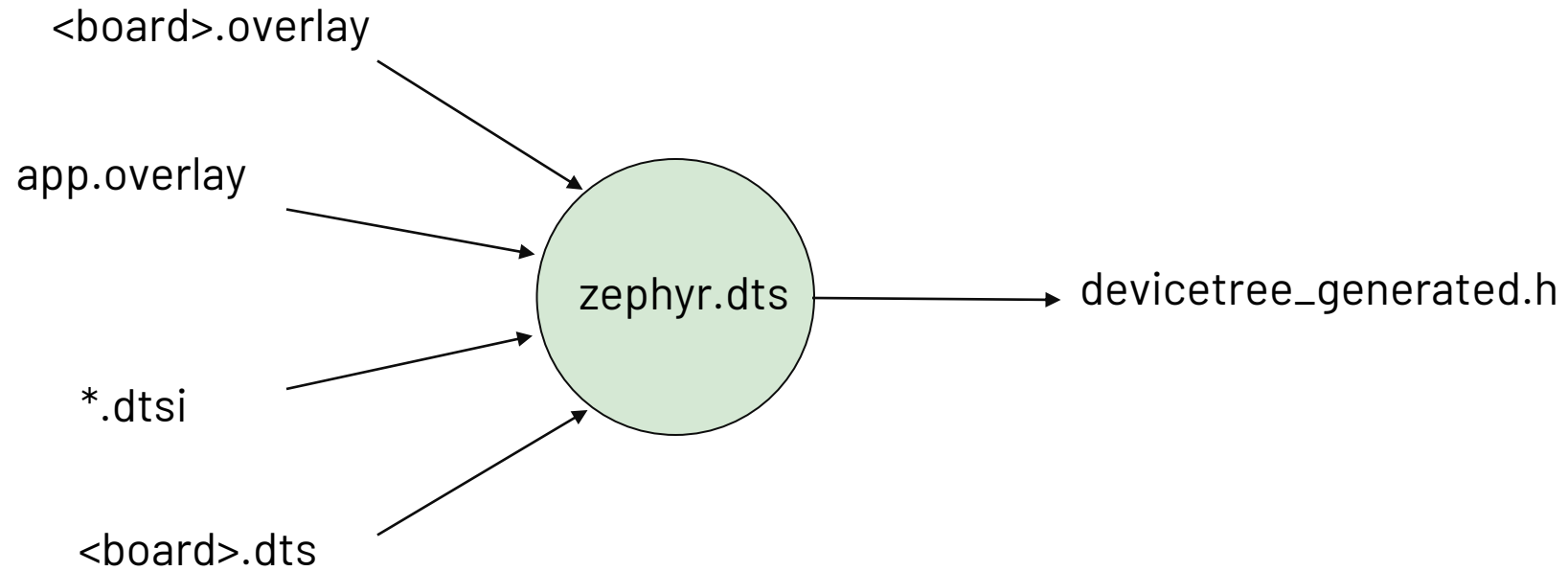
Specifies the **hardware** configuration in a hierarchical data structure

- Core
 - e.g., Arm Cortex-M4F, RISC-V
- SoC
 - Pins and clocks
 - Peripheral base addresses and interrupts
 - On-chip memories (flash, SRAM)
- Board
 - Off-chip devices (LEDs, buttons, sensors)
 - Pin headers (Arduino, Pmod, SDP)
- Application
 - Overlays

Devicetree Structure



Devicetree Inputs and Outputs



Devicetree Usage

zephyr.dts

```
uart2: serial@40044000 {
    compatible = "adi,max32-uart";
    reg = < 0x40044000 0x1000 >;
    clocks = < &gcr 0x1 0x1 >;
    clock-source = < 0x2 >;
    interrupts = < 0x22 0x0 >;
    status = "okay";
    pinctrl-0 = < &uart2a_tx_p1_10 &uart2a_rx_p1_9 >;
    pinctrl-names = "default";
    current-speed = < 0x1c200 >;
    data-bits = < 0x8 >;
    parity = "none";
};
```

devicetree_generated.h

```
#define DT_N_NODELABEL_uart2      DT_N_S_soc_S_serial_40044000
#define DT_N_INST_0_adi_max32_uart DT_N_S_soc_S_serial_40044000

#define DT_N_S_soc_S_serial_40044000_REG_NUM 1
#define DT_N_S_soc_S_serial_40044000_REG_IDX_0_EXISTS 1
#define DT_N_S_soc_S_serial_40044000_REG_IDX_0_VAL_ADDRESS 1074020352 /* 0x40044000 */
#define DT_N_S_soc_S_serial_40044000_REG_IDX_0_VAL_SIZE 4096 /* 0x1000 */

#define DT_N_S_soc_S_serial_40044000_P_current_speed 115200
#define DT_N_S_soc_S_serial_40044000_P_current_speed_EXISTS 1
```

sample_code.c

```
#define UART_NODE_W_LABEL DT_NODELABEL(uart2)
#define UART_NODE_W_INST DT_INST(0, adi_max32_uart)

DT_REG_ADDR(UART_NODE_W_LABEL)
DT_REG_SIZE(UART_NODE_W_LABEL)

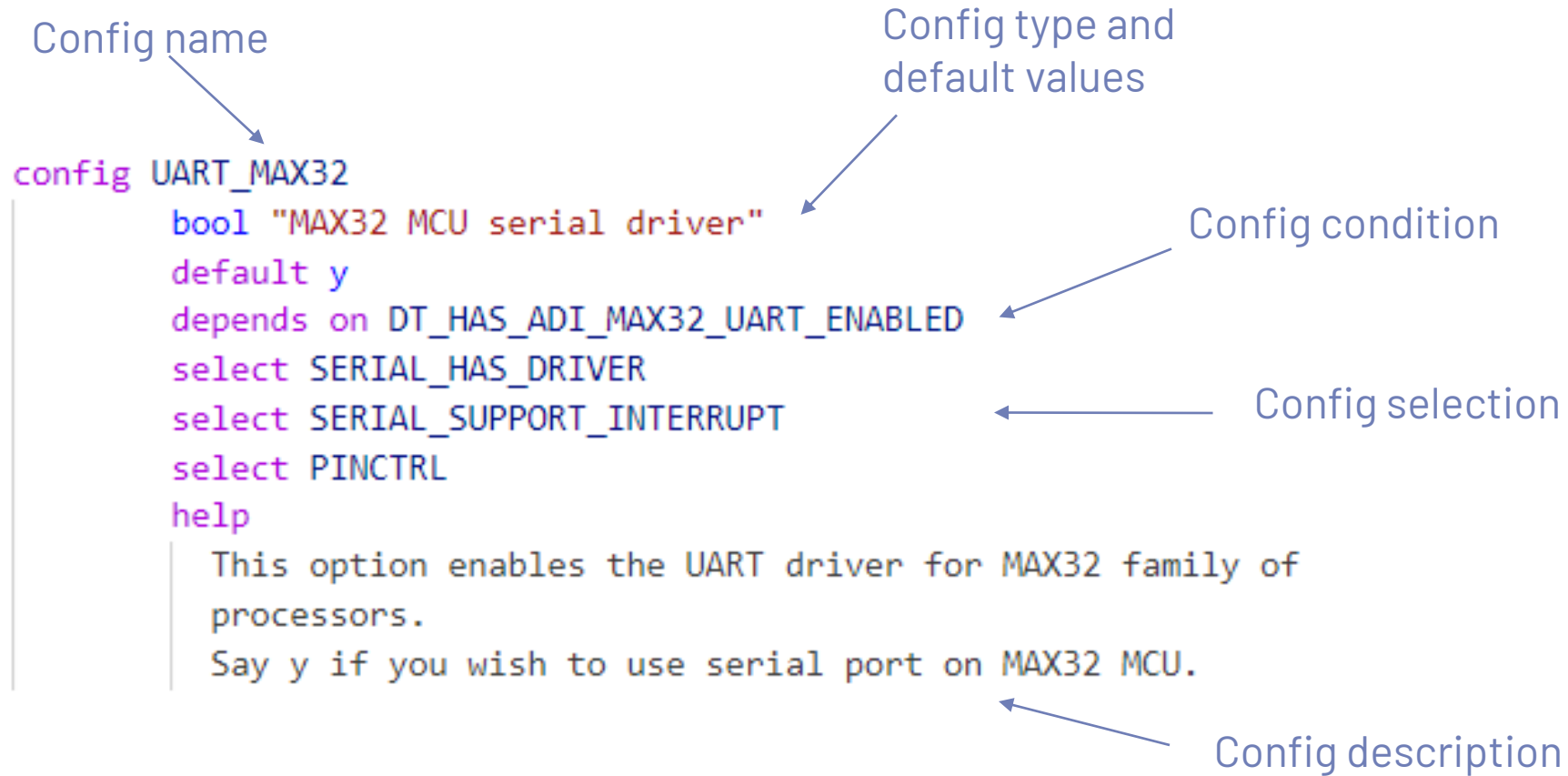
DT_PROP(UART_NODE_W_LABEL, current_speed)
DT_PROP(UART_NODE_W_INST, current_speed)
```


Kconfig

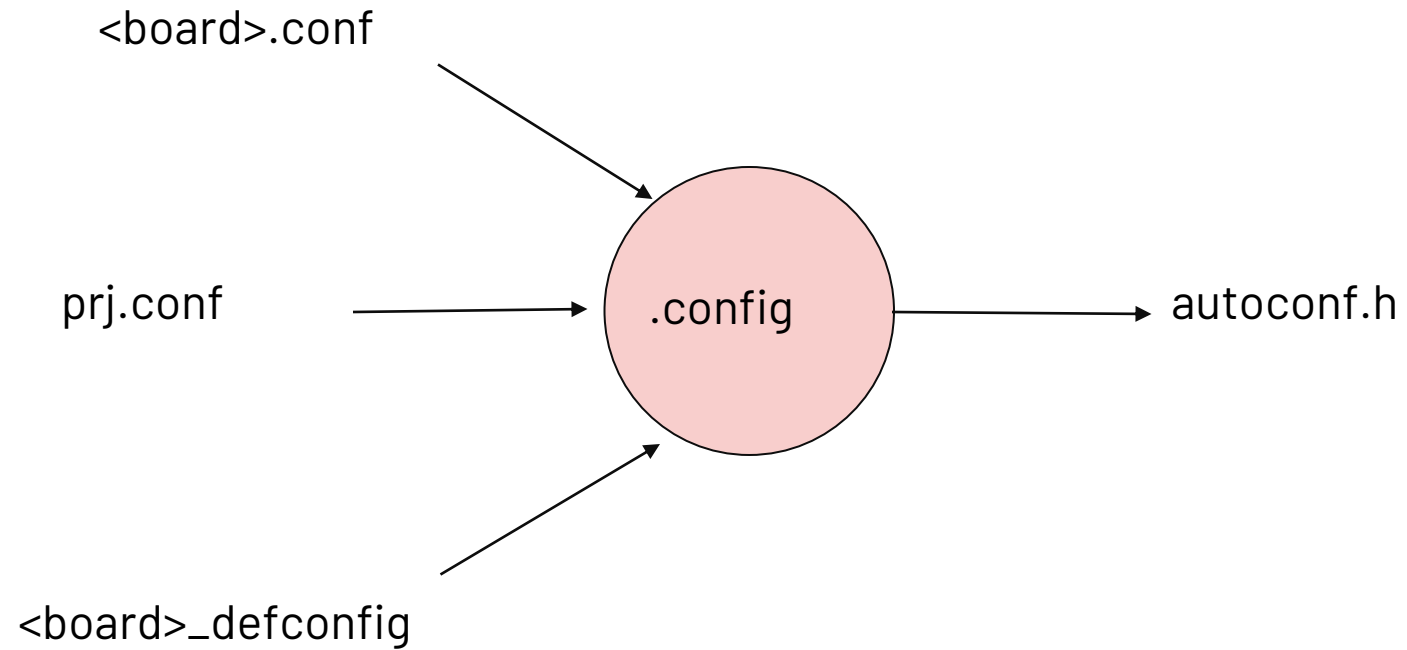
Specifies the **software** configuration

- Enable, disable, and configure software components and features
 - Device drivers (UART, SPI, sensors, etc.)
 - Subsystems (logging, shell, networking, Bluetooth, etc.)
 - Kernel (stack protection, tick frequency, scheduling algorithm, userspace, etc.)
- Conditional compilation
 - Build enabled components and features only
- Set default values

Kconfig Structure



Kconfig Inputs and Outputs



Kconfig Usage

.config

```
CONFIG_SERIAL=y
CONFIG_UART_MAX32=y

CONFIG_DT_HAS_ADI_MAX32_UART_ENABLED=y

CONFIG_SERIAL_HAS_DRIVER=y
CONFIG_SERIAL_SUPPORT_INTERRUPT=y
CONFIG_PINCTRL=y
```

zephyr/CMakeLists.txt

```
add_subdirectory(drivers)
```

zephyr/drivers/CMakeLists.txt

```
add_subdirectory_ifdef(CONFIG_SERIAL serial)
```

zephyr/drivers/serial/CMakeLists.txt

```
zephyr_library_sources_ifdef(CONFIG_UART_MAX32 uart_max32.c)
```

autoconf.h

```
#define CONFIG_SERIAL 1
#define CONFIG_UART_MAX32 1

#define CONFIG_DT_HAS_ADI_MAX32_UART_ENABLED 1

#define CONFIG_UART_INTERRUPT_DRIVEN 1
#define CONFIG_PINCTRL 1
#define CONFIG_SERIAL_HAS_DRIVER 1
```

zephyr/drivers/uart_max32.c

```
struct max32_uart_data {
#ifdef CONFIG_UART_INTERRUPT_DRIVEN
    uart_irq_callback_user_data_t cb; /* Interrupt callback */
    void *cb_data; /* Interrupt callback arg */
    uint32_t flags; /* Cached interrupt flags */
    uint32_t status; /* Cached status flags */
#endif
#ifdef CONFIG_UART_ASYNC_API
    struct max32_uart_async_data async;
#endif
    struct uart_config conf; /* baudrate, stopbits, ... */
};
```

CMakeLists.txt in Zephyr

Zephyr modules and CMakefiles should be accessible

- `find_package()`

Application specific source files should be added

- `target_sources()`

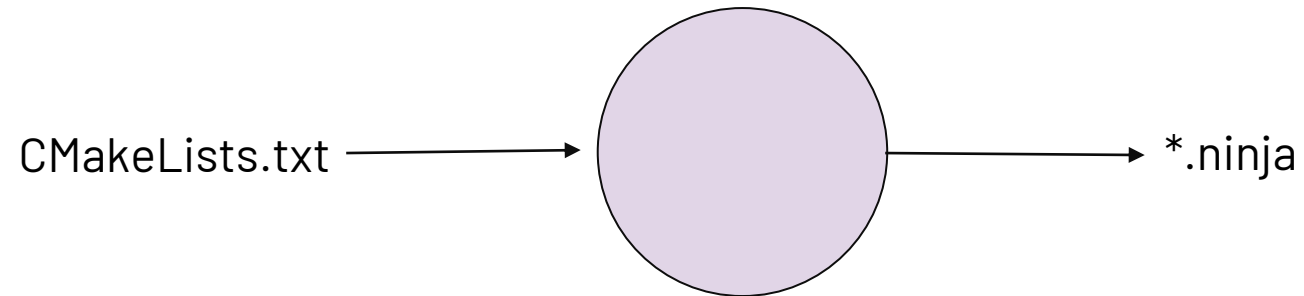
CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
```

```
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})  
project(hello_world)
```

```
target_sources(app PRIVATE src/main.c)
```

CMakeLists.txt in Zephyr



Build Phase

Separate Build

- Configuration Phase
 - `mkdir build && cd build`
 - `cmake -GNinja -DBOARD=apard32690/max32690/m4 ../samples/hello_world`
- Build Phase
 - `ninja`

Complete Build with One Command

- `west build -p -b apard32690/max32690/m4 samples/hello_world`

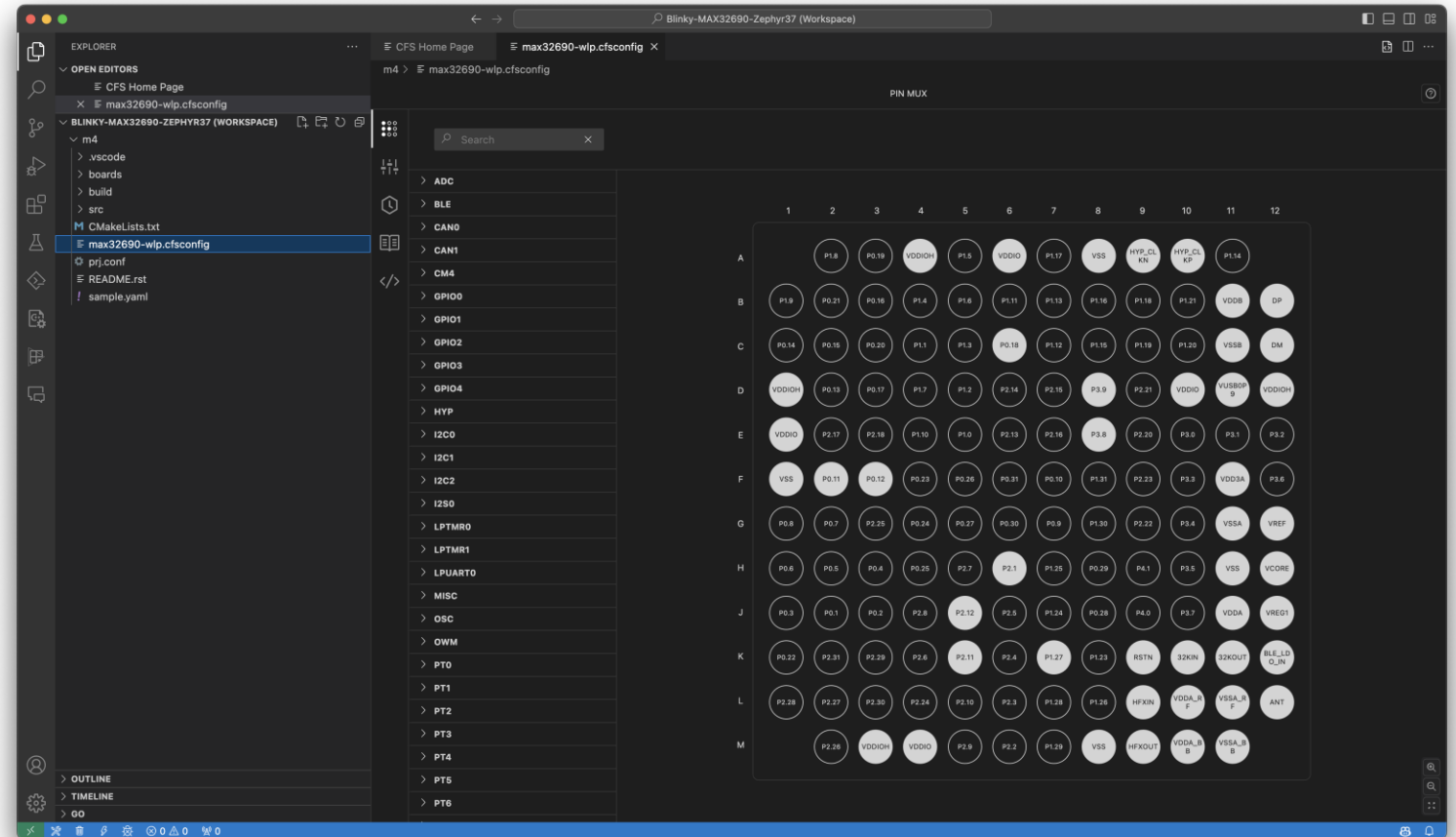


CodeFusion Studio Config Tools

Open the Config Tools (.cfsconfig)

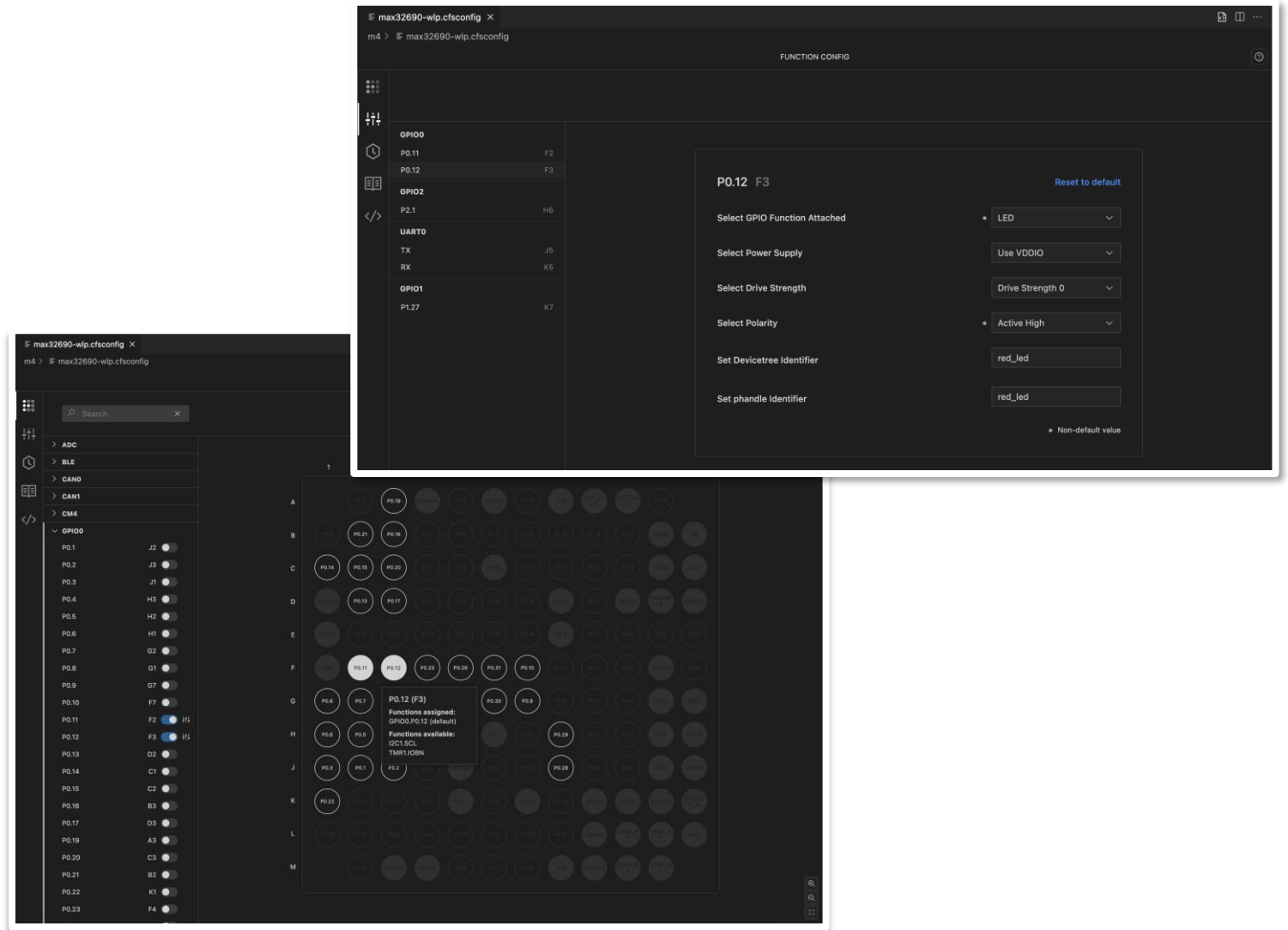
Config Tools are available by double-clicking on the ***.cfsconfig** file in your source tree.

1. Switch to the **Explorer** tab
2. Double-click on the **max32690-wlp.cfsconfig** file



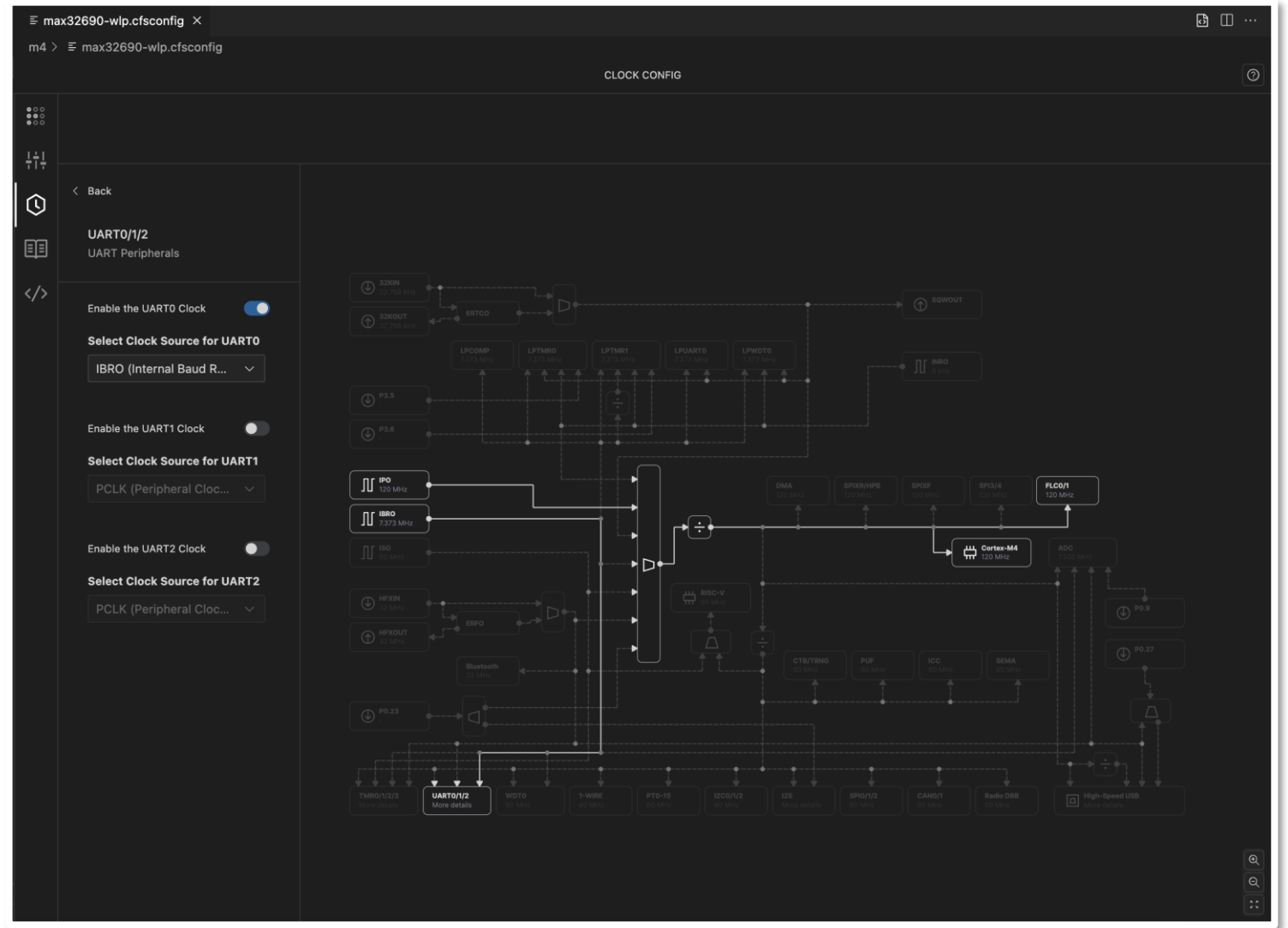
CodeFusion Studio Pin Config

Pin Config allows you to assign functions to a pin or pad (**pin mux**), and configure assigned functions (**pin config**).



CodeFusion Studio Clock Config

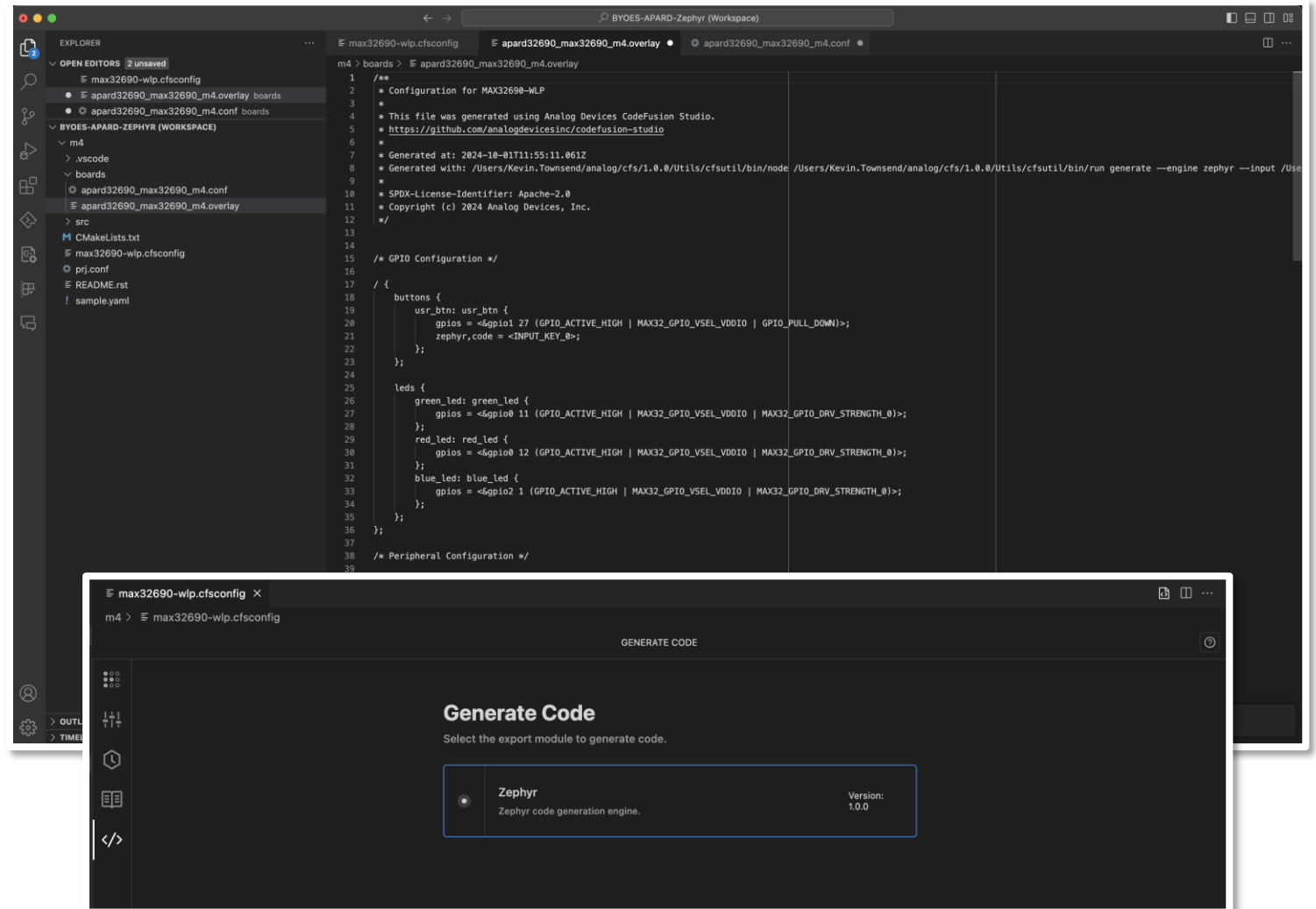
Clock Config is used to enable, disable or configure nodes in the clock tree. You can click on nodes in the clock tree to preview frequency, etc.



CodeFusion Studio Code Generation

CodeFusion Studio persists all config choices to a single **JSON file**, (*.cfsconfig).

This file is parsed by cfsutil to generate code for your firmware platform (Zephyr, MSDK, etc.).



Lab 2: Blinky and Devicetree Overlays

Lab 2: Blinky and Devicetree Overlays

Objectives

- Generate a Zephyr devicetree overlay with the CodeFusion Studio pin config tool
- Configure which LED is used in the blinky application by modifying a devicetree overlay

Observations

- Zephyr build system automatically picks up devicetree overlays that follow a specific naming convention based on path and board name
- CodeFusion Studio pin config tool generates devicetree overlays that follow the naming convention
- No C code is modified

Create a Project

Create a new project with these settings

Create a project

Ensure you have the details of the SoC at hand. Our project wizard guides you through the steps to create your new project.

Project name

Processor

Board

☒ Standard
 ☐ Custom

Processor Package: **WLP** ⓘ

Firmware Platform

Template

Project Location

☒ Use default location

Generate a Devicetree Overlay

Open the CodeFusion Studio config file

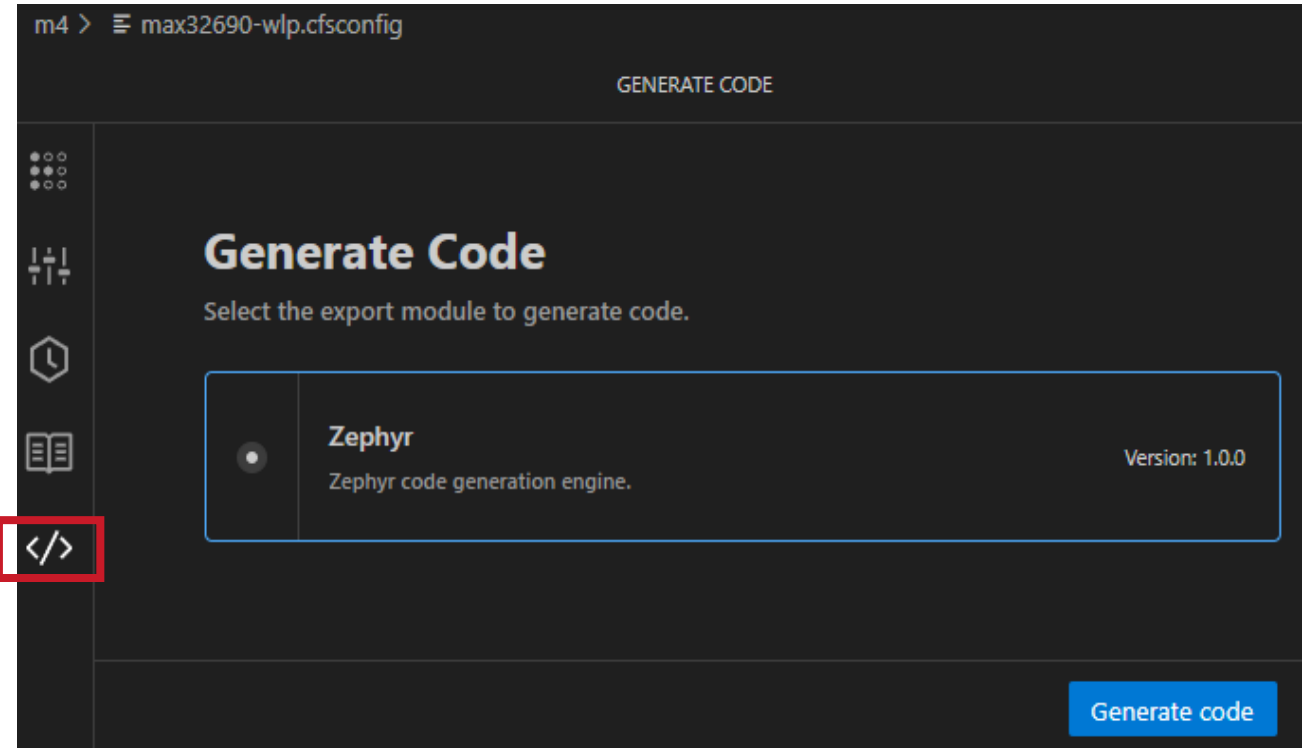
- max323690-wlp.cfsconfig

Use default settings in all CodeFusion Studio config sections:

- Pin Mux
- Function Config
- Clock Config
- Registers

In the CodeFusionStudio Generate Code section, click the “Generate code” button to generate files:

- boards/apard32690_max32690_m4.overlay
- boards/apard32690_max32690_m4.conf



Set the LED Alias

Open the generated devicetree overlay file:

- boards/apard32690_max32690_m4.overlay

Add an aliases section to the overlay and save the file

```
m4 > boards >  apard32690_max32690_m4.overlay
17 / {
23 };
24
25 leds {
26     green_led: green_led {
27         gpios = <&gpio0 11 (GPIO_ACTIVE_HIGH | MAX32_GPIO_VSEL_VDDIO | MAX32_GPIO_DRV_STRENGTH_0)>;
28     };
29     red_led: red_led {
30         gpios = <&gpio0 12 (GPIO_ACTIVE_HIGH | MAX32_GPIO_VSEL_VDDIO | MAX32_GPIO_DRV_STRENGTH_0)>;
31     };
32     blue_led: blue_led {
33         gpios = <&gpio2 1 (GPIO_ACTIVE_HIGH | MAX32_GPIO_VSEL_VDDIO | MAX32_GPIO_DRV_STRENGTH_0)>;
34     };
35 };
36
37     aliases {
38         led0 = &green_led;
39     };
40 };
```

The blinky application uses the led0 alias

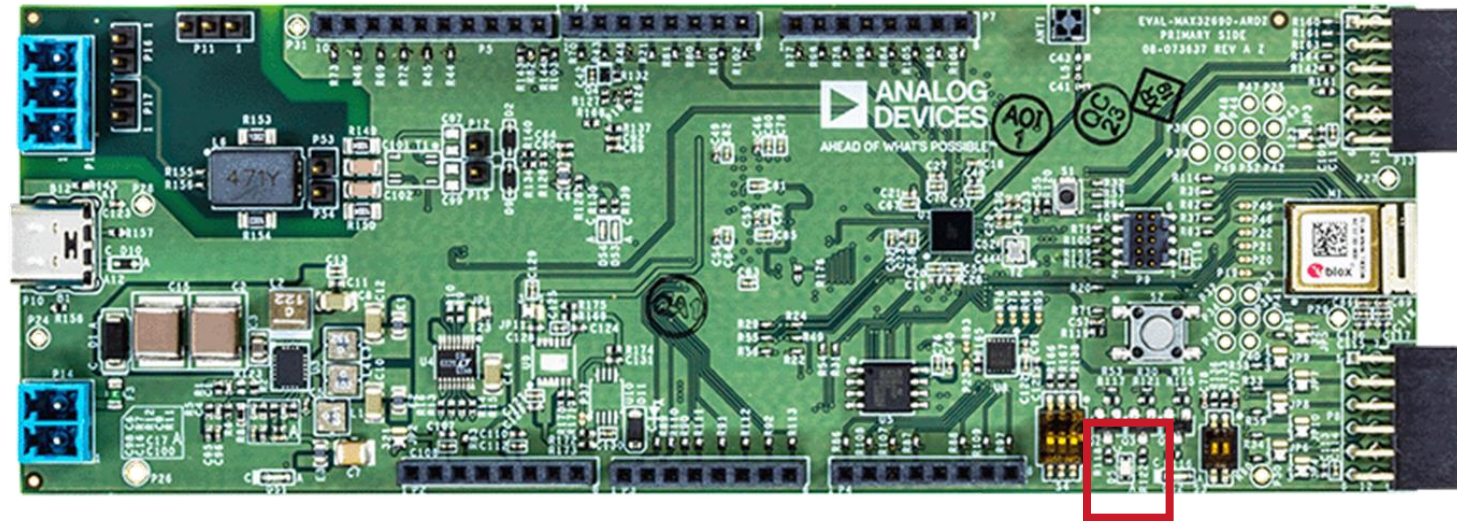
```
m4 > src >  main.c > ...
11 // 1000 mSEC = 1 sec
12 #define SLEEP_TIME_MS 1000
13
14 /* The devicetree node identifier for the "led0" alias. */
15 #define LED0_NODE DT_ALIAS(led0)
16
17 /*
18  * A build error on this line means your board is unsupported.
19  * See the sample documentation for information on how to fix this.
20  */
21 static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
22
23 int main(void)
```

Build, Flash, and Blink

Build the application

Program the application into flash

See the LED blink



Break! (15 minutes)

Lab 3: Debug

Lab 3: Debug

- **3.1 Basic debug features in the IDE**

- Starting a debug sessions
- Breakpoints
- Stepping through code
- Viewing registers and variables

- **3.2 Advanced debug features in the IDE**

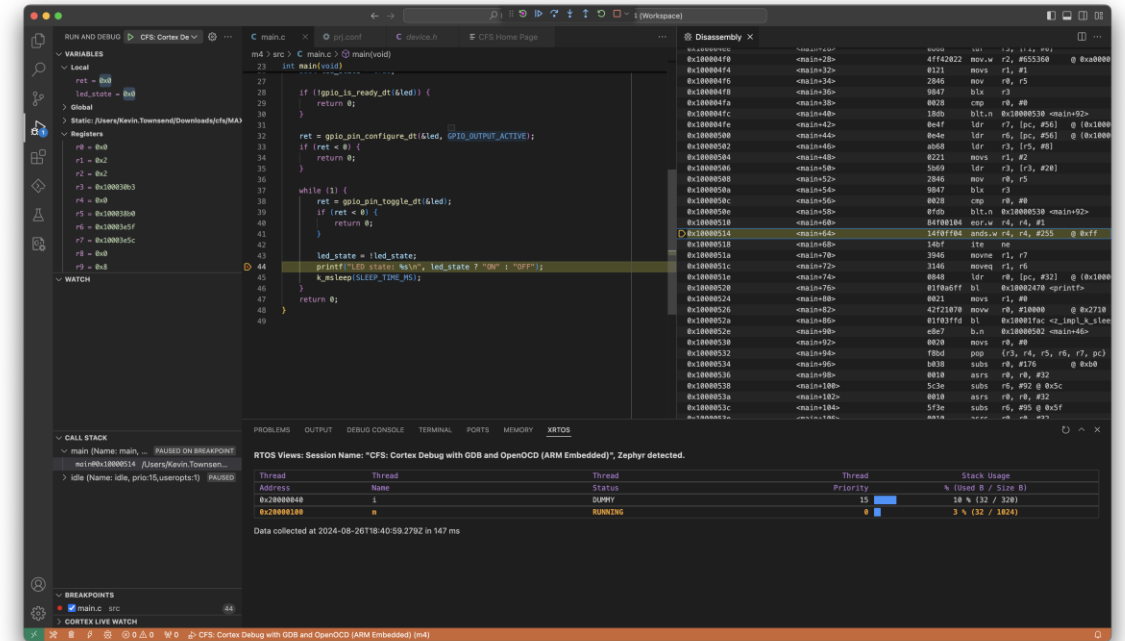
- Conditional Breakpoints
- Working with Variables
- Disassembly View

- **3.3 Call Stacks and Thread Awareness**

- Stack Backtrace
- RTOS Thread Monitor

Lab 3.1

Basic Debug Features in the IDE

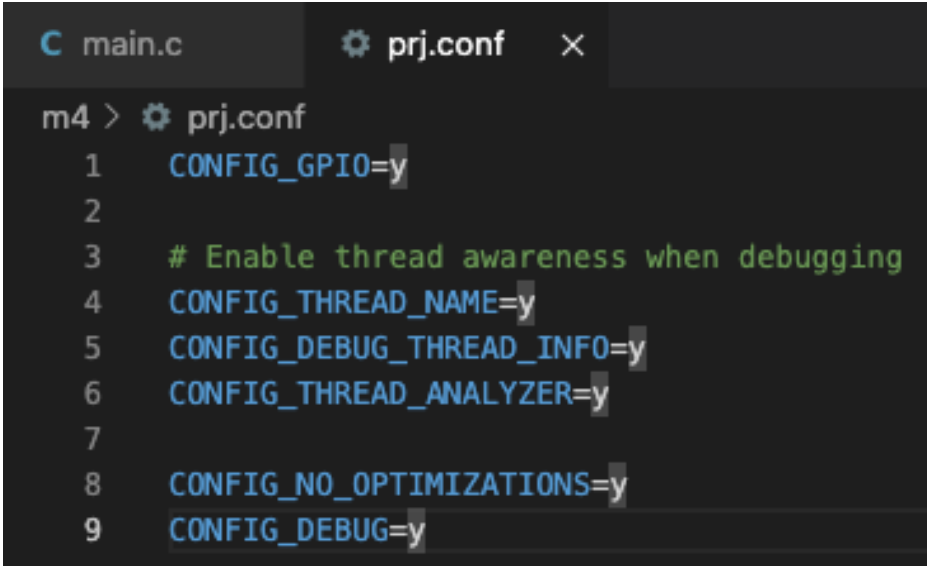


Lab 3.1: Disable Optimizations

If compiler optimizations are enabled by default – which they often are – stepping through your code can behave unexpectedly.

If possible, it's better to disable compiler optimizations when debugging, which can be done by adding the following lines to your **prj.conf** file:

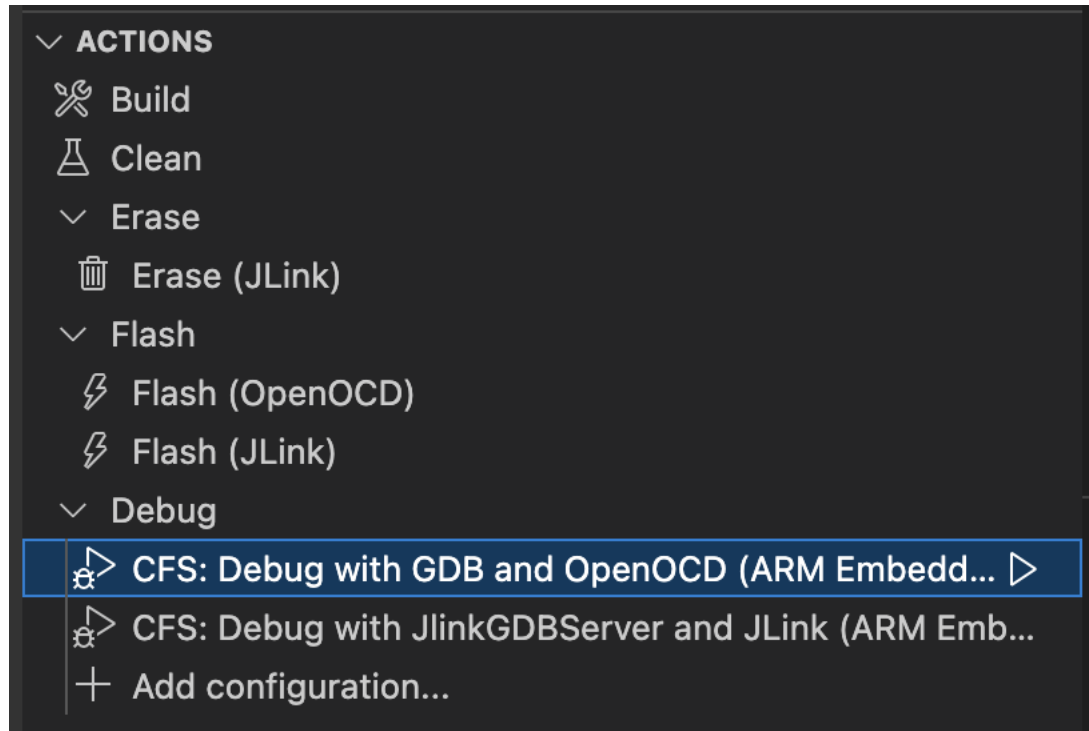
- `CONFIG_DEBUG=y`
- `CONFIG_NO_OPTIMIZATIONS=y`



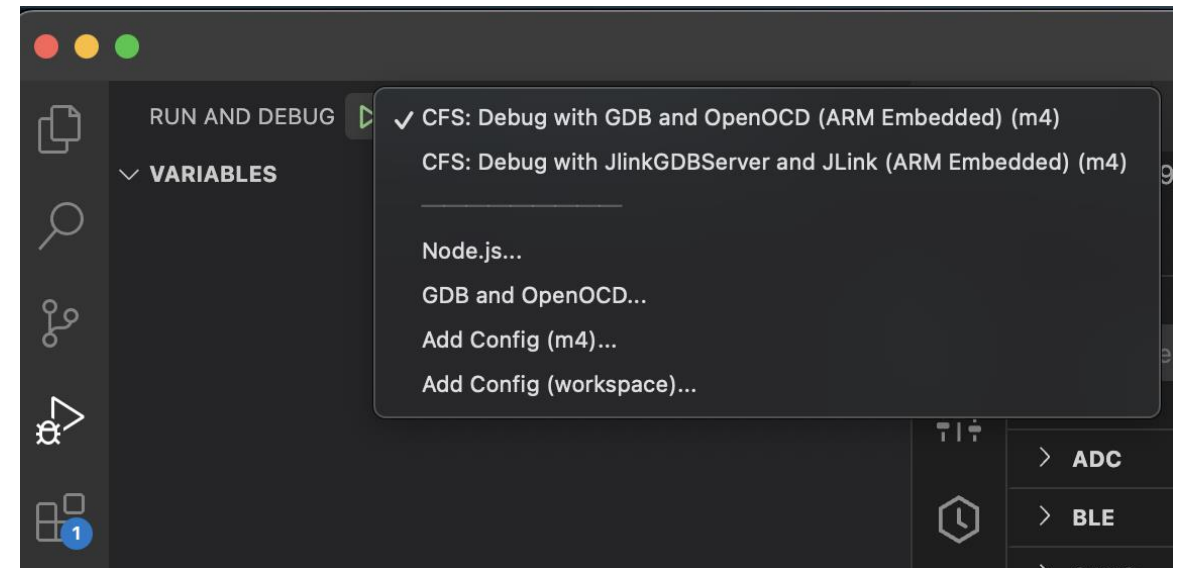
```
C main.c  prj.conf x
m4 > prj.conf
1  CONFIG_GPIO=y
2
3  # Enable thread awareness when debugging
4  CONFIG_THREAD_NAME=y
5  CONFIG_DEBUG_THREAD_INFO=y
6  CONFIG_THREAD_ANALYZER=y
7
8  CONFIG_NO_OPTIMIZATIONS=y
9  CONFIG_DEBUG=y
```

Lab 3.1: Starting a Debug Session

Option A: Using the CodeFusion Studio Panel



Option B: Using the Run and Debug Panel



Lab 3.1: Basic Breakpoints

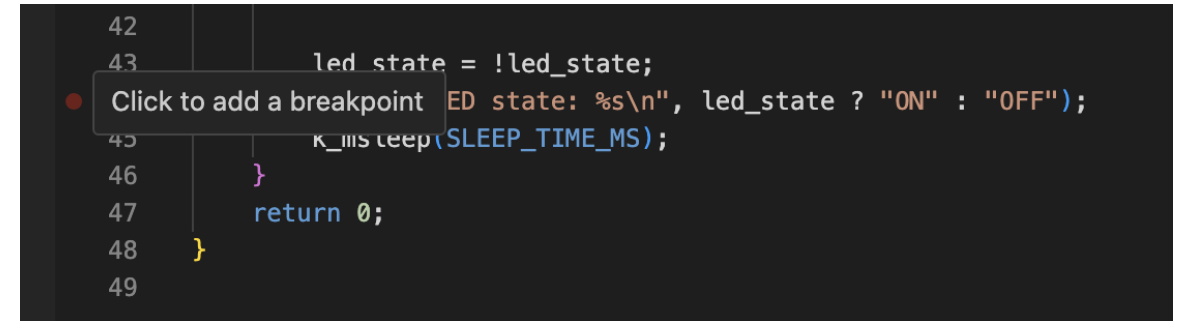
Description

Breakpoints cause the core to **'halt'** at a given location.

When you hit a breakpoint, you can check the system state, and perform some of the following actions:

- Step through the code from that location
- View/Modify Variables
- View/Modify Core Registers
- View/Modify Memory
- View the Call Stack

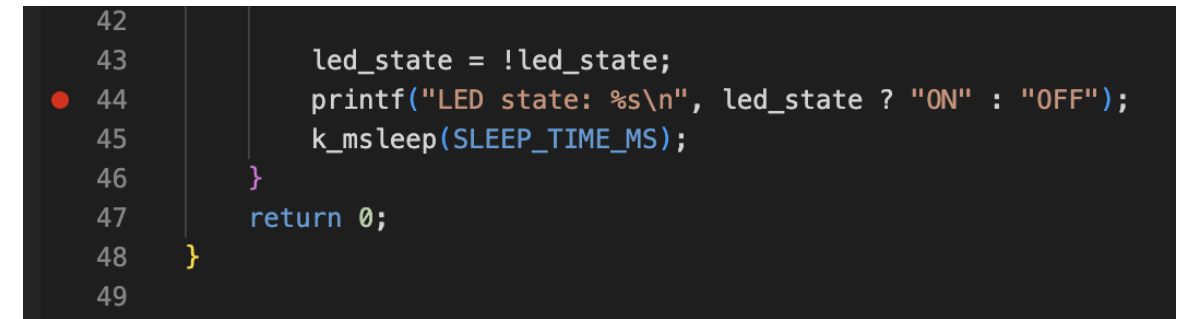
Setting and Viewing Breakpoints in the IDE



```

42
43     led_state = !led_state;
44     printf("LED state: %s\n", led_state ? "ON" : "OFF");
45     k_msleep(SLEEP_TIME_MS);
46 }
47 return 0;
48 }
49

```

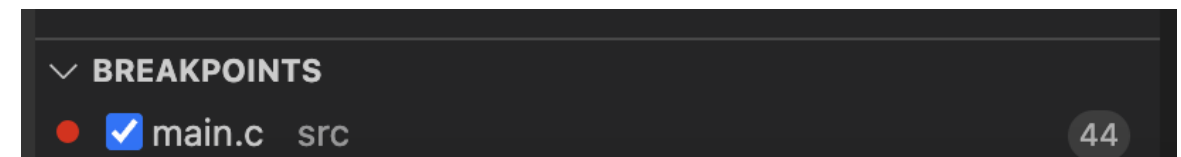


```

42
43     led_state = !led_state;
44     printf("LED state: %s\n", led_state ? "ON" : "OFF");
45     k_msleep(SLEEP_TIME_MS);
46 }
47 return 0;
48 }
49

```

View/Toggle breakpoints in the **Run and Debug** panel:



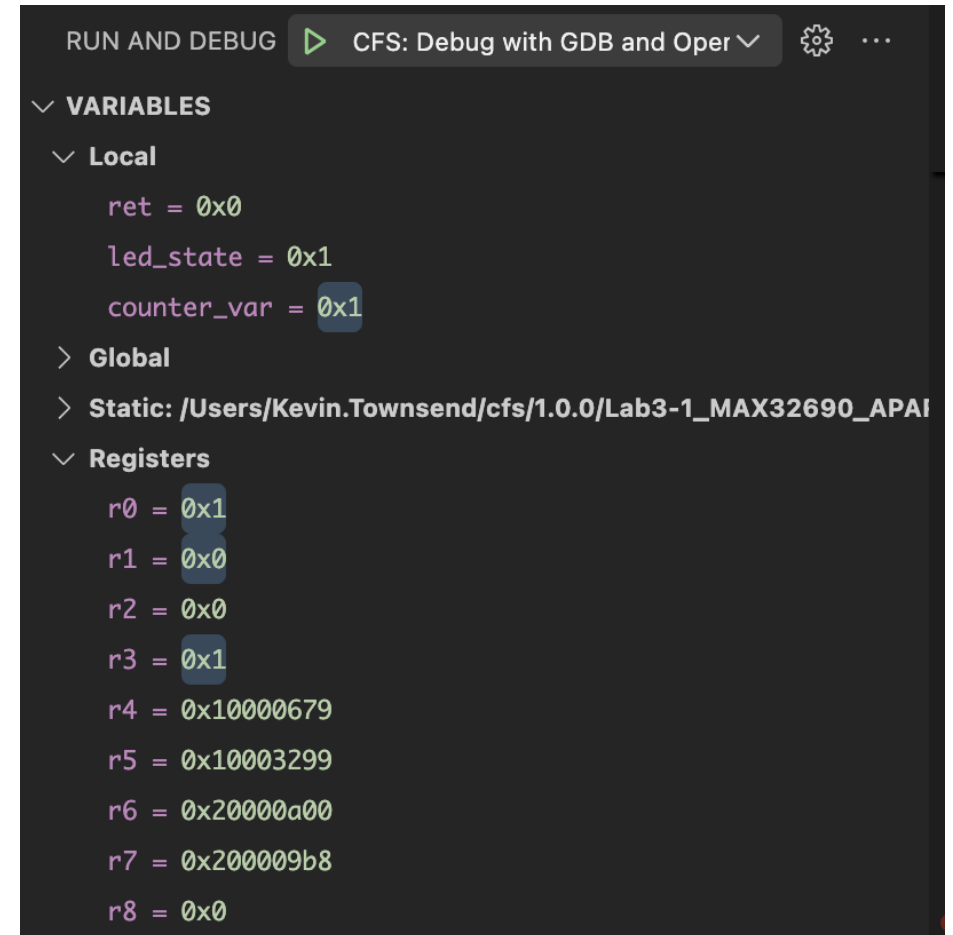
Lab 3.1: Stepping Through Code



- **Reset** the Device () ————— ↑
- **Continue** Execution (F5) ————— ↑
- **Step Over** Current Function (F10) ————— ↑
- **Step Into** Current Function (F11) ————— ↑
- **Step Out** of Current Function (Shift + F11) ————— ↑
- **Restart** the Application (Shift + F5) ————— ↑
- **Stop** Debugging (F5) ————— ↑

Lab 3.1: Registers and Variables

- Whenever you hit a breakpoint, you can see any **local** or **global** registers, as well as the **registers** that are part of the core being debugged.
- If a value changes between the previous known state, and the updated state and the BP, the value will be highlighted.



The screenshot shows a debugger window with the following content:

```

RUN AND DEBUG  ▶ CFS: Debug with GDB and Oper  ⚙️ ...
└─ VARIABLES
  └─ Local
    ret = 0x0
    led_state = 0x1
    counter_var = 0x1
  > Global
  > Static: /Users/Kevin.Townsend/cfs/1.0.0/Lab3-1_MAX32690_APAL
  └─ Registers
    r0 = 0x1
    r1 = 0x0
    r2 = 0x0
    r3 = 0x1
    r4 = 0x10000679
    r5 = 0x10003299
    r6 = 0x20000a00
    r7 = 0x200009b8
    r8 = 0x0
  
```

Lab 3.1: Lab Exercise

Build and Flash the Sample

1. Copy the source code from **lab31.txt** to **main.c** and save the file in the IDE
2. Add the following to your **prj.conf** file:
 - CONFIG_DEBUG=y
 - CONFIG_NO_OPTIMIZATIONS=y

```

3  # Enable thread awareness when debugging
4  CONFIG_THREAD_NAME=y
5  CONFIG_DEBUG_THREAD_INFO=y
6  CONFIG_THREAD_ANALYZER=y
7
8  CONFIG_NO_OPTIMIZATIONS=y
9  CONFIG_DEBUG=y

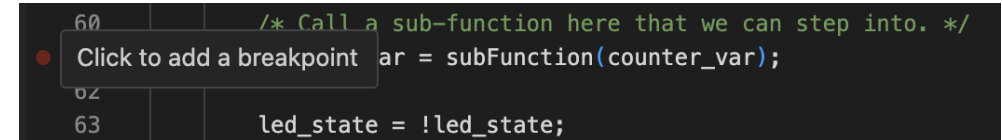
```

3. Click **CFS Quick Actions > Clean**
4. Click **CFS Quick Actions > Build**
5. Click **CFS Quick Actions > Flash (OpenOCD)**

Set breakpoints and debug

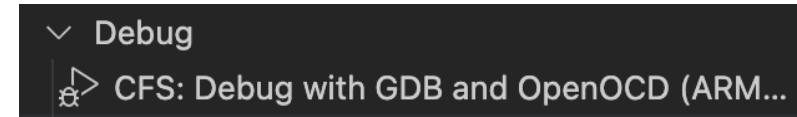
- Set a breakpoint on **line 61** of **main.c**

```
counter_var = subFunction(counter_var);
```

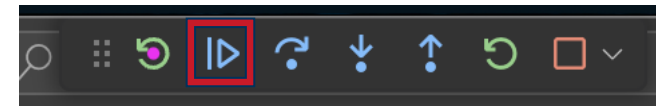


- **CFS Quick Actions > Debug with GDB and OpenOCD**

(Hint: Click the 'Play' icon to the right)



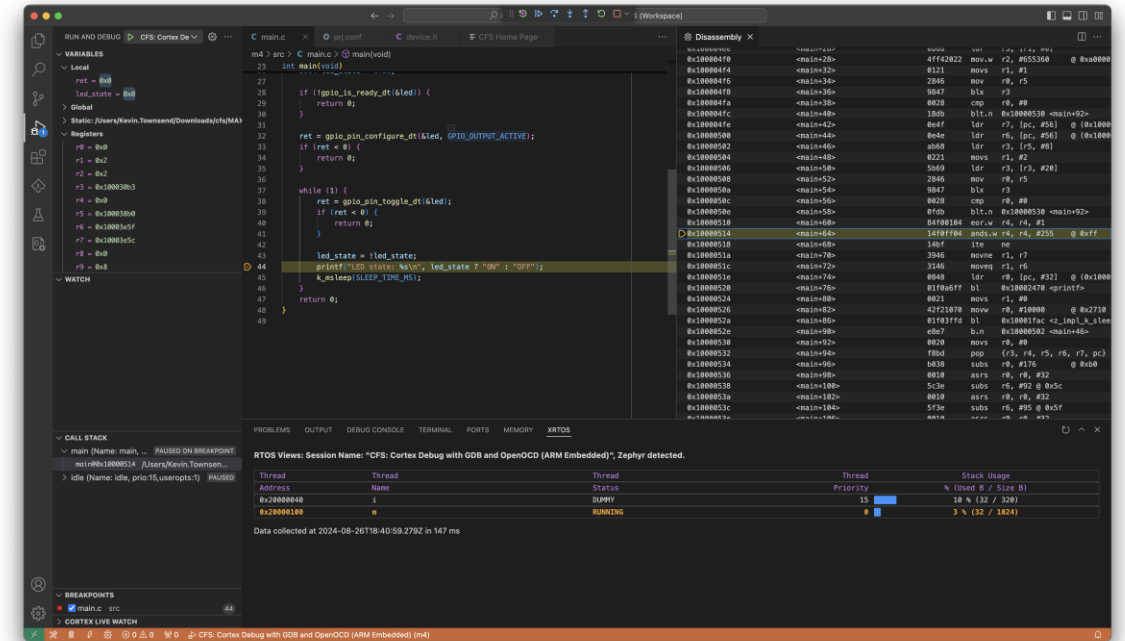
- Once the debugger starts, hit the **Continue** icon to continue execution until the first breakpoint



- **Step into** and **over** functions and examine variables

Lab 3.2

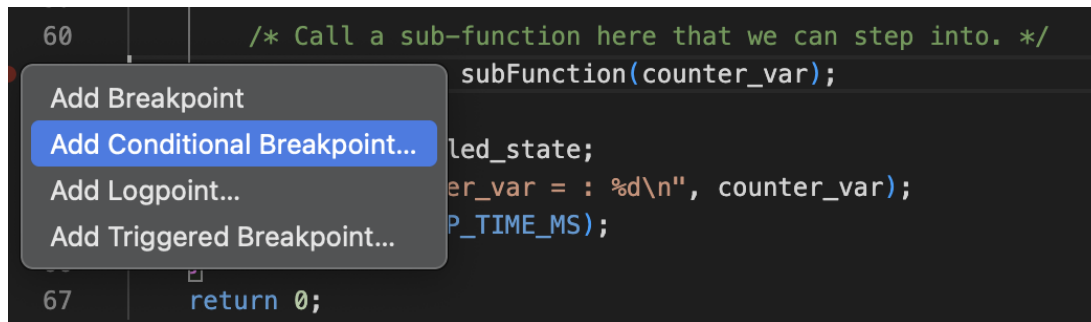
Advanced Debug Features in the IDE



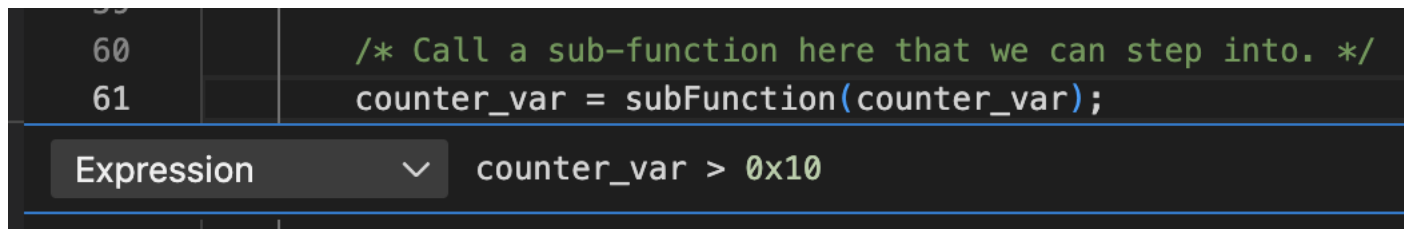
Lab 3.2: Conditional Breakpoints

You can also set **Conditional Breakpoints**. These BPs only trigger when a specific condition evaluates to true.

1. Right-click to create a breakpoint and select **Add Conditional Breakpoint ...**



2. Enter the conditions, such as "counter_var > 0x10"

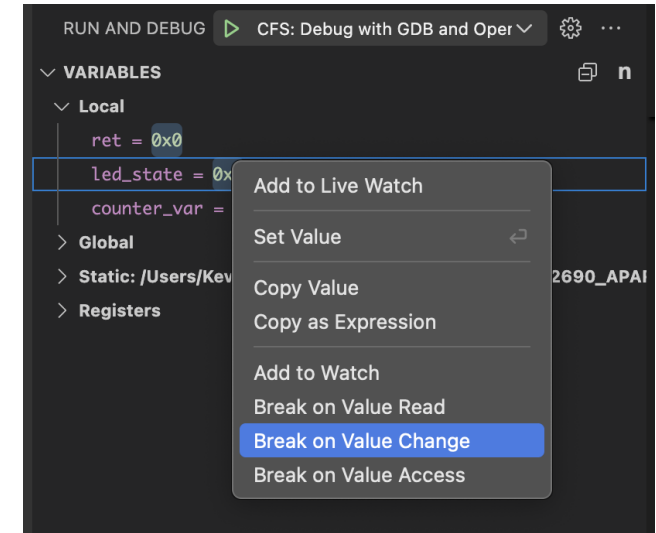


3. The breakpoint will only trigger when "counter_var" evaluates to greater than 0x10

Lab 3.2: Working with Variables

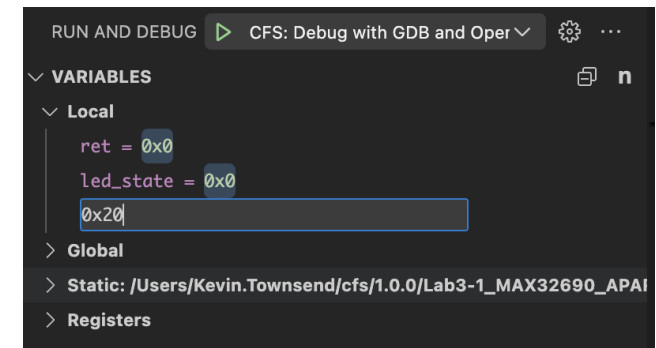
You can **trigger a breakpoint** when a variable is **read** or **changed**:

1. Right-click and select **Break on Value Read**
2. Right-click and select **Break on Value Change**



You can also manually change a variable from inside a breakpoint:

1. Right-click **Set Value** and enter the new value:



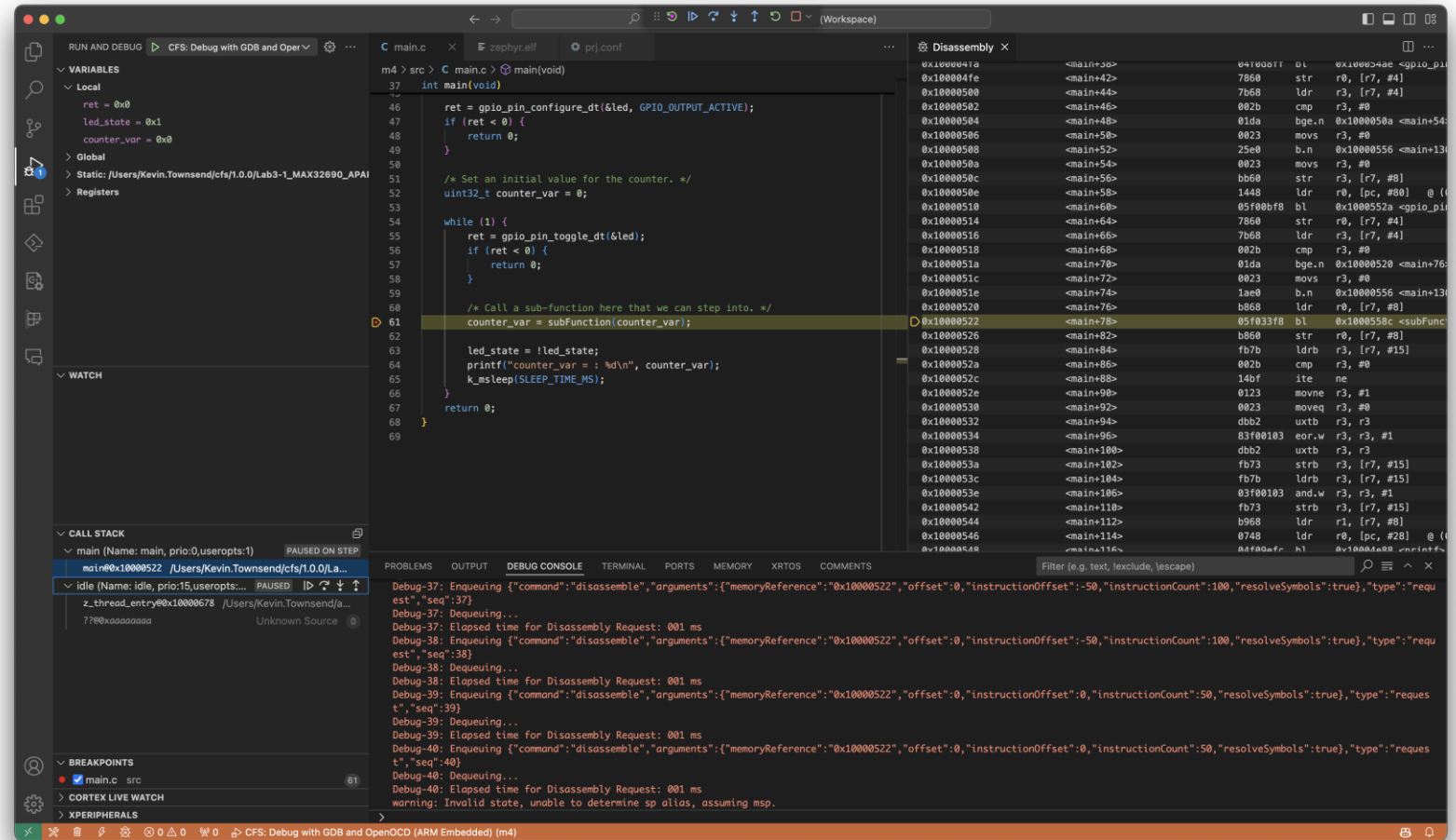
Lab 3.2: Disassembly View

While debugging:

- Right-click in the source editor and select **Open Disassembly View**:

Jump to Cursor
Open Disassembly View

- Source Navigation:
 - When focus is on the **C source file**, you single-step through the C code
 - When focus is on the **Assembly view**, you single-step through the ASM code



Lab 3.2: Lab Exercise

Debug the same sample, and:

1. Change the value of `counter_var` to 12
2. Set a conditional breakpoint on line 61 when `counter_var > 0x10`:

```
counter_var > 0x10
```
3. Continue running until you hit the debugger
4. Check the value of `counter_var`, which should be 0x11 or 17

Step through assembly code while at the breakpoint:

1. Right-click on the current line of code and click **Open Disassembly View**
2. Drag the disassembly tab over to the left-hand side of the IDE to view it side-by-side with the C code
3. Switch focus to the assembly view
4. Single step through a few assembly instructions

Lab 3.3: Stack Backtrace

- Each thread in an RTOS maintains it's own **call stack**, which records the history of function calls to explain **how you got to your current location in code**.
- The **Call Stack Panel** shows you this data for each thread:
 - Click on functions in the stack to navigate to source
 - You can also see the **state** of each stack

```

✓ CALL STACK
  ✓ main (Name: main, prio:0,useropts:1) PAUSED
    __ISB@0x100022ba /Users/Kevin.Townsend/analog/cfs/...
    arch_irq_unlock@0x100022ba /Users/Kevin.Townsend/...
    arch_swap@0x100022ba /Users/Kevin.Townsend/analog...
    z_swap_irqlock@0x10007e24 /Users/Kevin.Townsend/a...
    z_swap@0x1000420a /Users/Kevin.Townsend/analog/cf...
    z_tick_sleep@0x1000420a /Users/Kevin.Townsend/ana...
    z_impl_k_sleep@0x10004294 /Users/Kevin.Townsend/a...
    k_sleep@0x100053f2 /Users/Kevin.Townsend/cfs/1.0.0/...
    k_msleep@0x100053d2 /Users/Kevin.Townsend/analog/...
    main@0x10000554 /Users/Kevin.Townsend/cfs/1.0.0/La...
  ✓ idle (Name: idle,... PAUSED ON USER REQUEST |▶ ↺ ⚙ ⬆ ⬆
    __ISB@0x10007848 /Users/Kevin.Townsend/analog/cfs/...
    arch_irq_lock@0x10007848 /Users/Kevin.Townsend/an...
    idle@0x10007848 /Users/Kevin.Townsend/analog/cfs/1...
    z_thread_entry@0x100006a0 /Users/Kevin.Townsend/a...

```

Lab 3.3: RTOS Thread Monitor

By enabling a few Zephyr config parameters, you can display the following thread information in the **XRTOS** tab:

- Base address for the thread
- The thread name
- Thread status (Running, Suspended, etc.)
- Thread Priority
- Stack Usage

To enable this feature, the following Kconfig flags must be set:

```
CONFIG_THREAD_NAME=y
CONFIG_DEBUG_THREAD_INFO=y
CONFIG_THREAD_ANALYZER=y
```

PROBLEMS

74

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

MEMORY

XRTOS

COMMENTS

RTOS Views: Session Name: "CFS: Debug with GDB and OpenOCD (ARM Embedded)", Zephyr detected.

Thread Address	Thread Name	Thread Status	Thread Priority	Stack Usage
				% (Used B / Size B)
0x20000098	main	SUSPENDED for: 10010 ms	0	36 % (368 / 1024)
0x20000158	idle	RUNNING	15	35 % (112 / 320)

Data collected at 2024-10-03T19:39:57.330Z in 153 ms

Lab 3.3: Lab Exercise

Call Stack Navigation

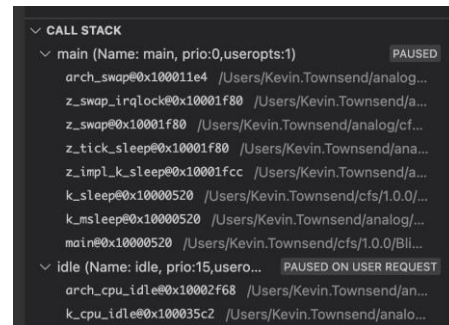
1. In the "Run and Debug" tab, clear any breakpoints:



2. If you're not already debugging, start a new debug session: **Debug with GDB and OpenOCD (Arm Embedded)**
3. Let the program run for a few seconds
4. Hit the **Pause Execution** button in the debug panel:



5. Navigate to the **Call Stack** panel in the "Run and Debug" tab:



RTOS Thread Awareness

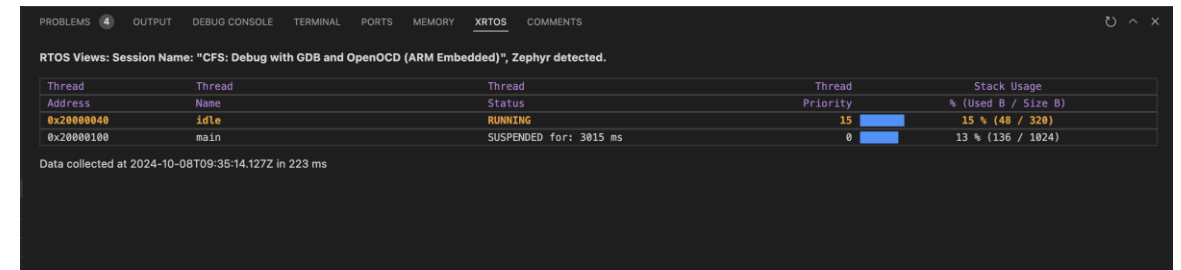
1. Make sure these flags are defined in prj.conf:

`CONFIG_THREAD_NAME=y`

`CONFIG_DEBUG_THREAD_INFO=y`

`CONFIG_THREAD_ANALYZER=y`

2. At a breakpoint, open the **XRTOS** tab, and you should see a list of threads and their state:



NOTE: You might need to continue execution then pause again once before the thread statistics have enough time to be read by the thread debug tool.

Instrumentation and Profiling Tools

Tracing Subsystem

Features

- Synchronous or asynchronous tracing
- Thread info and switched in/out events
- ISR enter/exit events

Formats

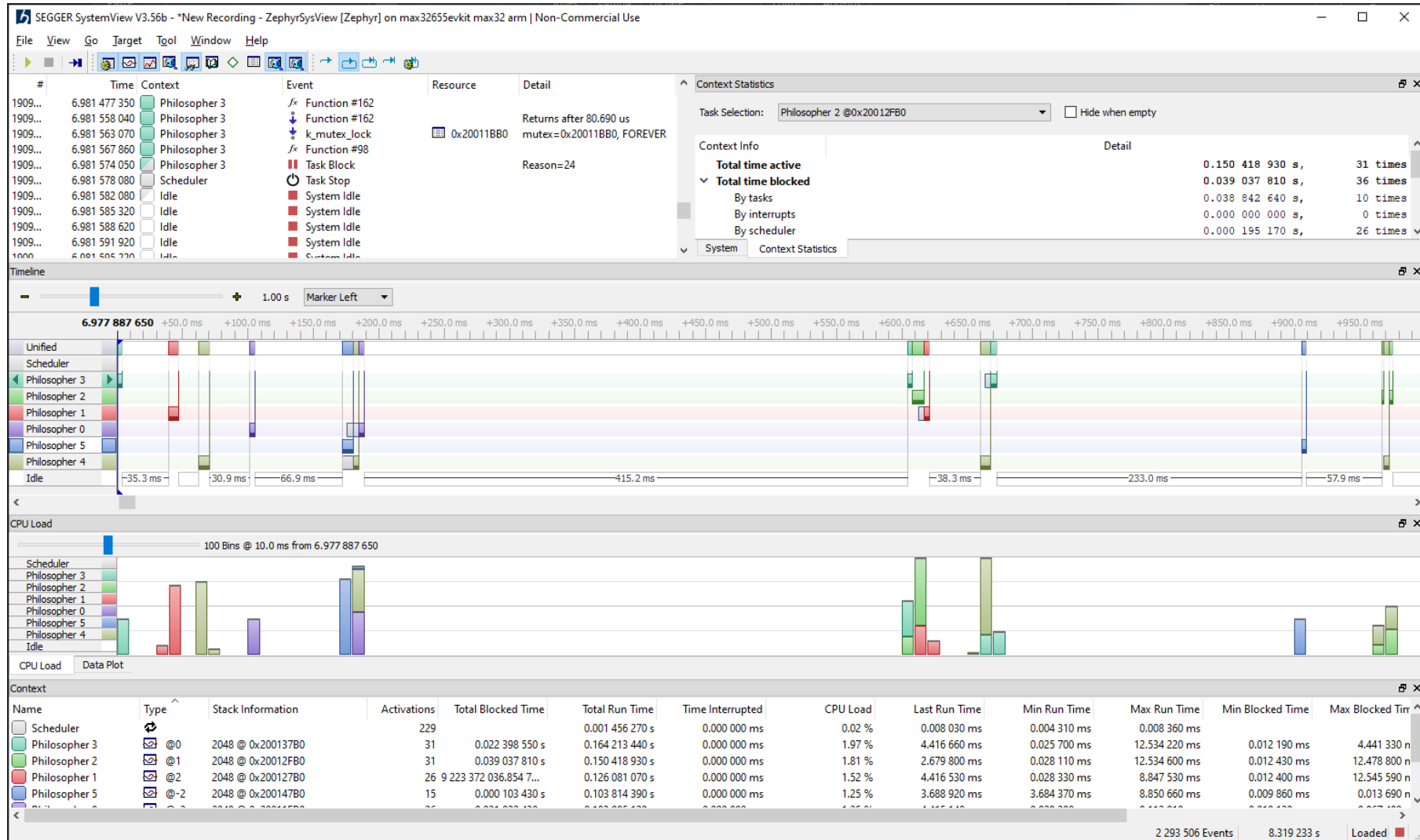
- Common Trace Format (CTF)
- SEGGER SystemView
- Percepio Tracealyzer
- User-defined, e.g., proprietary format or toggle GPIO

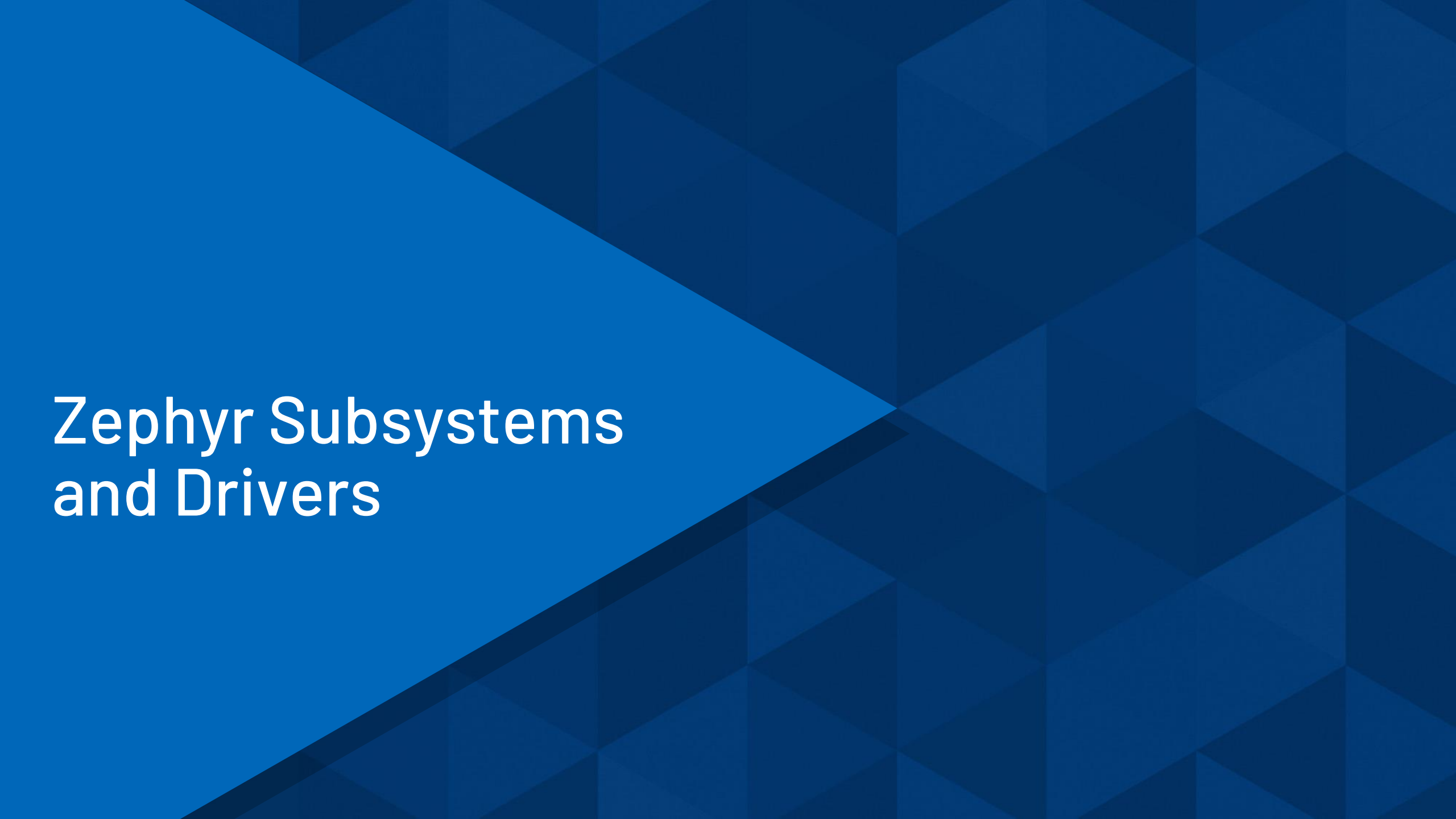
Backends (transports)

- UART, Segger RTT, POSIX, RAM

<https://docs.zephyrproject.org/3.7.0/services/tracing/index.html>

Tracing Subsystem with SEGGER SystemView





Zephyr Subsystems and Drivers

Peripheral Device Drivers

Each type of driver has a standard API

- GPIO, UART, I2C, SPI, ADC, DAC, flash, sensor, and more
- All hardware vendors implement the same API for each type of driver

Consumers of the standard API don't care which implementation is under the hood

- Therefore, off-chip drivers, such as sensors, are portable, and can be used with ADI and non-ADI MCUs

```
C: > analog > cfs > 1.0.0 > SDK > zephyr > zephyr > drivers > sensor > adi > adxl345 > C adxl345.c > adxl345_init(const device *)
45  stat struct spi_buf_set{(void).buffers = (const struct spi_buf *)buf, (void).count = (size_t)2U}
55  };
56
57  if (cmd == ADXL345_READ_CMD) {
58      tx.count = 1;
59      return spi_transceive_dt(&cfg->bus.spi, &tx, &rx);
60  } else {
61      return spi_write_dt(&cfg->bus.spi, &tx);
62  }
63 }
```

<https://docs.zephyrproject.org/3.7.0/kernel/drivers/index.html>

<https://docs.zephyrproject.org/3.7.0/hardware/peripherals/index.html>

Shell Subsystem

Features

- Static and dynamic commands
- Smart command completion with the Tab key
- ANSI escape codes for cursor control and color printing

Backends (transports)

- UART, Segger RTT, USB, telnet, RPMSG, MQTT

Modules

- Kernel, sensor, I2C, SPI, GPIO, USB, flash, file system, networking, Bluetooth, and more
- Boards and applications can register custom commands

```
uart:~$ kernel
kernel - Kernel commands
Subcommands:
  cycles   : Kernel cycles.
  stacks   : List threads stack usage.
  threads  : List kernel threads.
  uptime   : Kernel uptime. Can be called with the -p or --pretty options
  version  : Kernel version.
  sleep    : ms
uart:~$ kernel version
Zephyr version 3.7.99
uart:~$ device list
devices:
- clock-controller@40000000 (READY)
- gpio@4000c000 (READY)
- gpio@40080400 (READY)
- gpio@4000a000 (READY)
- gpio@40009000 (READY)
- gpio@40008000 (READY)
- serial@40042000 (READY)
- spi@400be400 (READY)
- adxl345@0 (READY)
uart:~$
```

<https://docs.zephyrproject.org/3.7.0/services/shell/index.html>

Logging Subsystem

Features

- Immediate or deferred logging
- String or dictionary-based logging
- Compile time filtering on the module level
- Runtime filtering on the module instance level
- Four severity levels: error, warning, info, and debug

Backends (transports)

- UART, Segger RTT, shell, BLE, file system, IPC service, Arm SWO

```
uart:~$ sensor get adxl345@0 accel_xyz
channel type=3(accel_xyz) index=0 shift=4 num_samples=1 value=17844360000ns, (0.306456, -0.919373, 8.274359)
uart:~$
[00:02:58.445,000] <dbg> sensor_compat: sensor_submit_fallback_sync: value[0]=0.306457, q[0]@0x200020b8=41131962, shift: 4
[00:02:58.446,000] <dbg> sensor_compat: sensor_submit_fallback_sync: value[1]=-0.919373, q[1]@0x200020bc=-123396156, shift: 4
[00:02:58.446,000] <dbg> sensor_compat: sensor_submit_fallback_sync: value[2]=8.274360, q[2]@0x200020c0=1110565799, shift: 4
[00:02:58.446,000] <dbg> sensor_compat: sensor_submit_fallback_sync: Total channels in header: 3
[00:02:58.446,000] <dbg> sensor_shell: sensor_shell_processing_callback: decoding 1 frames from channel accel_xyz:0
```

<https://docs.zephyrproject.org/3.7.0/services/logging/index.html>

Shield Boards

Extend a base board with one or more daughter boards

- Devicetree overlay specifies hardware components on the shield
- Standard nodelabels are defined by base boards and used by shield boards
 - `arduino_i2c`, `arduino_spi`, `arduino_serial`, `pmod_spi`

Activate by adding `--shield` arguments to the west command

- `west build -b apard32690/max32690/m4 --shield pmod_acl`

```
C: > analog > cfs > 1.0.0 > SDK > zephyr > zephyr > boards > shields > pmod_acl > pmod_acl.overlay
6
7  &pmod_spi {
8      status = "okay";
9
10     adxl345_pmod_acl: adxl345@0 {
11         compatible = "adi,adxl345";
12         reg = <0x0>;
13         spi-max-frequency = <DT_FREQ_M(1)>;
14         status = "okay";
15     };
16 };
```

```
C: > analog > cfs > 1.0.0 > SDK > zephyr > zephyr > boards > adi > apard32690 > apard32690_max32690_m4.dts
129 pmod_spi: &spi4 {
130     pinctrl-0 = <&spi4a_miso_p1_2 &spi4a_mosi_p1_1 &spi4a_sck_p1_3
131     &spi4a_ss0_p1_0>;
132     pinctrl-names = "default";
133 };
```

<https://docs.zephyrproject.org/3.7.0/hardware/porting/shields.html>
<https://docs.zephyrproject.org/3.7.0/boards/index.html#shields>

Lab 4: Sensors, Shell and Shields

Lab 4: Sensors, Shell, and Shields

Objectives

- Enable Zephyr sensor drivers, logging and shell subsystems
- Generate a Zephyr devicetree overlay with the CodeFusion Studio pin config tool
- Enable a sensor shield board in the build task

Observations

- Shell commands allow runtime interaction with sensor drivers and the kernel
- Logging subsystem provides useful debug information
- Shield boards enable extending base boards with additional functionality

Create a Project

Create a new project with these settings

Create a project

Ensure you have the details of the SoC at hand. Our project wizard guides you through the steps to create your new project.

Project name

lab4-sensors-shell-shields

Processor

MAX32690

Board

☒ Standard ☐ Custom

AD-APARD32690-SL

Processor Package: **WLP** ⓘ

Firmware Platform

Zephyr 3.7

Template

blinky

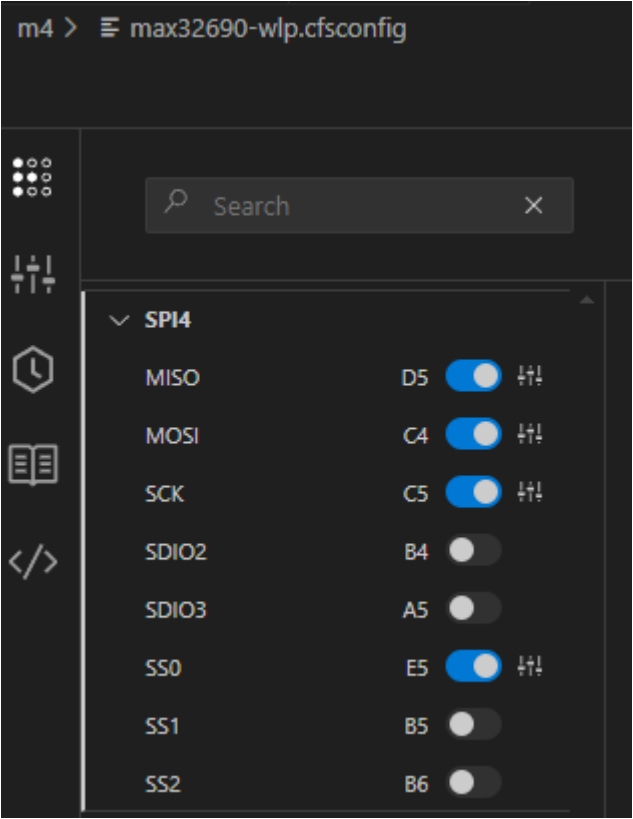
Project Location

☒ Use default location

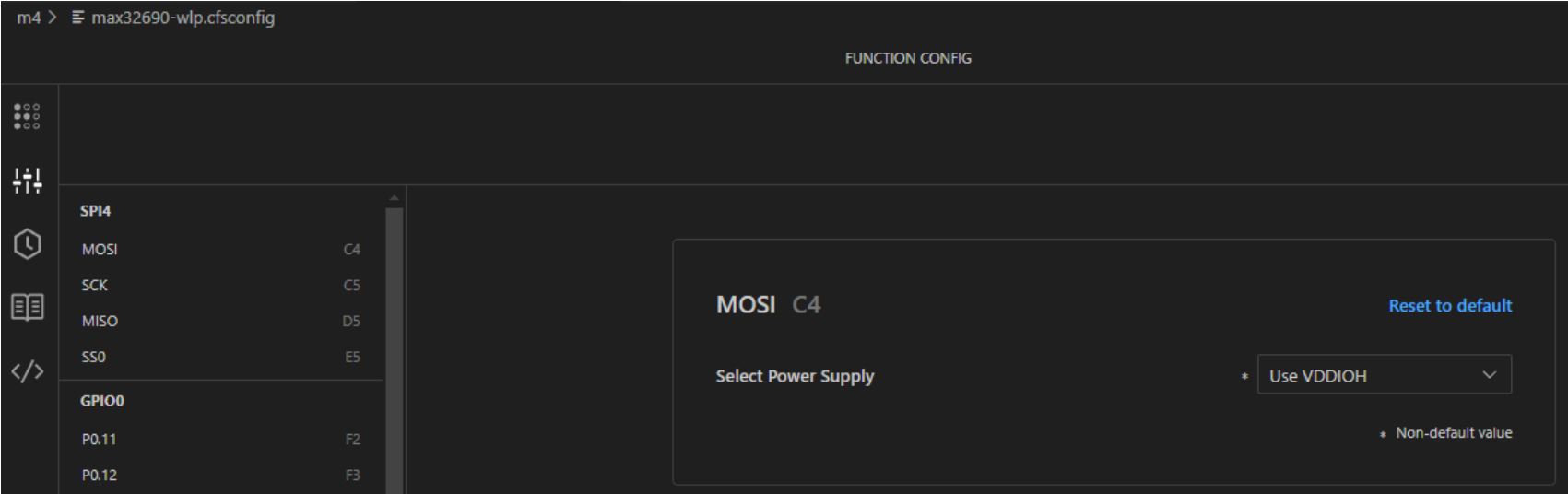
C:/Users/mhelm/cfs/1.0.0

Browse

Configure SPI Pins and Generate a Devicetree Overlay

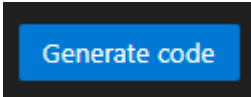


Enable SPI4 MOSI, MISO, SCK, and SS0 pins



Select SPI4 MOSI, MISO, SCK, and SS0 power supply to **Use VDDIOH**

Generate a devicetree overlay

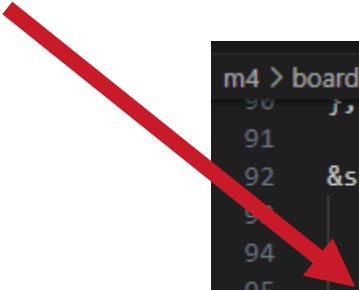


Enable the SPI Instance

Open the generated devicetree overlay file:

- boards/apard32690_max32690_m4.overlay

Change the spi4 node status to "okay" and save the file



```
m4 > boards >  apard32690_max32690_m4.overlay
90 },
91
92 &spi4 {
93     pinctrl-0 = <&spi4a_mosi_p1_1 &spi4a_sck_p1_3 &spi4a_miso_p1_2 &spi4a_ss0_p1_0>;
94     pinctrl-names = "default";
95     status = "okay";
96 };
```

Enable Subsystems and Sensor Driver Features

Open the application conf file:

- prj.conf

Add Kconfigs and save the file

```
m4 > ⚙ prj.conf
1  CONFIG_GPIO=y
2
3  # Enable thread awareness when debugging
4  CONFIG_THREAD_NAME=y
5  CONFIG_DEBUG_THREAD_INFO=y
6  CONFIG_THREAD_ANALYZER=y
7
8  CONFIG_LOG=y
9  CONFIG_SHELL=y
10 CONFIG_SENSOR=y
11 CONFIG_SENSOR_INFO=y
12 CONFIG_SENSOR_SHELL=y
13 CONFIG_SENSOR_LOG_LEVEL_DBG=y
```

Modify the C Application

Open the C application file:

- src/main.c

Comment out or delete the printf line and save the file

- This avoids mixing printf output with shell output

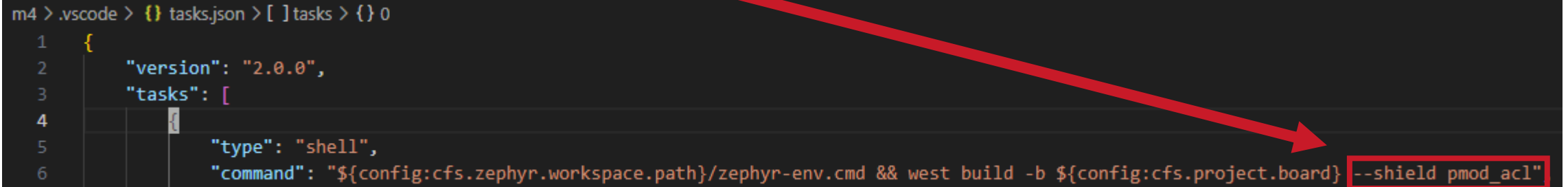
```
m4 > src > C main.c > ...
23  int main(void)
36
37      while (1) {
38          ret = gpio_pin_toggle_dt(&led);
39          if (ret < 0) {
40              return 0;
41          }
42
43          led_state = !led_state;
44          /* printf("LED state: %s\n", led_state ? "ON" : "OFF"); */
45          k_msleep(SLEEP_TIME_MS);
46      }
47      return 0;
48  }
```

Modify the Build Task

Click the Terminal menu and select "Configure Tasks"

Select the "CFS: build" task

Add the shield argument to the command

A screenshot of a VS Code editor showing the tasks.json file. The file is open in a terminal window. The content of the file is as follows:

```
m4 > .vscode > {} tasks.json > [ ] tasks > {} 0
1  {
2    "version": "2.0.0",
3    "tasks": [
4      {
5        "type": "shell",
6        "command": "${config:cfs.zephyr.workspace.path}/zephyr-env.cmd && west build -b ${config:cfs.project.board} --shield pmod_acl"
```

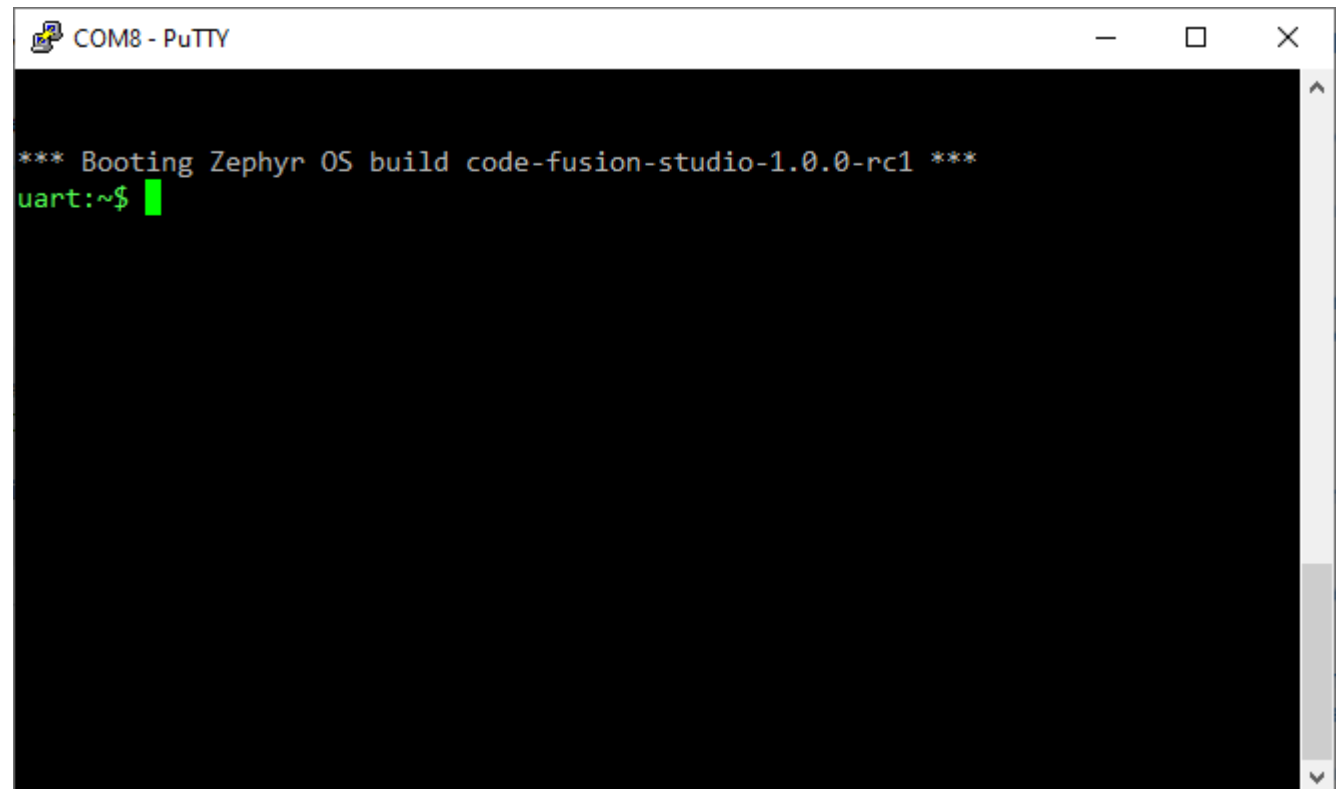
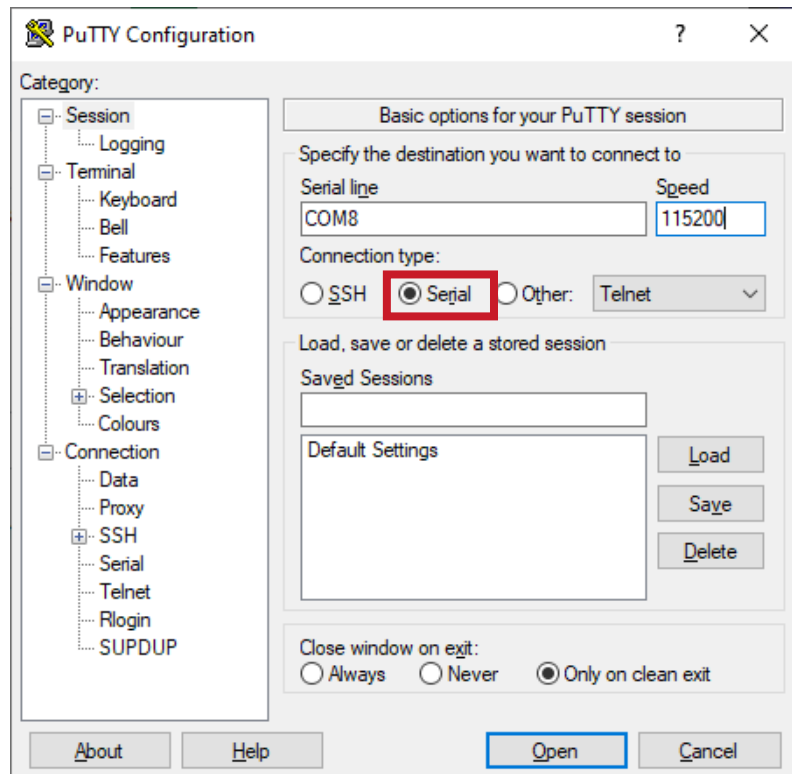
A red arrow points from the text "Add the shield argument to the command" to the `--shield pmod_acl` argument in the command string. The argument is highlighted with a red box.

Build, Flash, and Open a Serial Connection

Build the application

Program the application into flash

Open a PuTTY serial connection



Try Some Shell Commands

```
uart:~$ sensor info
device name: adxl345@0, vendor: Analog Devices, Inc., model: adxl345, friendly name: (null)
uart:~$ sensor get adxl345@0 accel_xyz
channel type=3(accel_xyz) index=0 shift=4 num_samples=1 value=121976700000ns, (0.306456, -0.919373, 8.274359)
[00:02:01.979,000] <dbg> sensor_compat: sensor_submit_fallback_sync: value[0]=0.306457, q[0]@0x200020b8=41131962, shift: 4
[00:02:01.979,000] <dbg> sensor_compat: sensor_submit_fallback_sync: value[1]=-0.919373, q[1]@0x200020bc=-123396156, shift: 4
[00:02:01.979,000] <dbg> sensor_compat: sensor_submit_fallback_sync: value[2]=8.274360, q[2]@0x200020c0=1110565799, shift: 4
[00:02:01.979,000] <dbg> sensor_compat: sensor_submit_fallback_sync: Total channels in header: 3
[00:02:01.979,000] <dbg> sensor_shell: sensor_shell_processing_callback: decoding 1 frames from channel accel_xyz:0
uart:~$ kernel version
Zephyr version 3.7.99
uart:~$
uart:~$ kernel threads
Scheduler: 638 since last call
Threads:
0x20000cc0
  options: 0x0, priority: 0 timeout: 0
  state: pending, entry: 0x10001da9
  Total execution cycles: 1132928 (0 %)
  stack size 2048, unused 1576, usage 472 / 2048 (23 %)

*0x20000918 shell_uart
  options: 0x0, priority: 14 timeout: 0
  state: queued, entry: 0x100041f5
  Total execution cycles: 5560684 (0 %)
  stack size 2048, unused 648, usage 1400 / 2048 (68 %)
```

AHEAD OF WHAT'S POSSIBLE

analog.com

